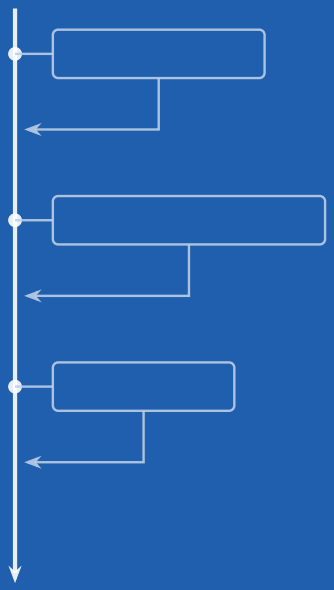
BİTİRME PROJESİ · TEKNİK RAPOR

# minGPT ve FPGA Üzerinde Tamsayı Çıkarım Hızlandırıcısı

Transformer matematiğinden bit düzeyinde doğrulanmış SystemVerilog donanımına



**Murat Atceken**

Ekim 2026

| | |
|---|---|
| Model | minGPT `gpt-nano` — 3 katman, 3 dikkat başı, $C = 48$, 85 728 parametre |
| Aritmetik | INT8 ağırlık ve aktivasyon, INT16 artık akış, yalnızca tamsayı |
| Doğruluk | 729 / 729 girdi doğru; RTL, referans modelle 8 019 adımda bit-exact |
| Performans | 14 544 çevrim/token — 100 MHz'de 145 µs |
| Hedef kart | PYNQ-Z2 (Zynq-7020, xc7z020clg400-1) |
| Kaynak kod | `github.com/Muratatceken/mingpt-fpga` |

## Özet

Bu çalışmada Andrej Karpathy'nin minGPT modelinin `gpt-nano` yapılandırması, yalnızca tamsayı aritmetiği kullanan bir FPGA hızlandırıcısı olarak SystemVerilog ile gerçeklenmiştir. Model; 3 katman, 3 dikkat başı ve 48 boyutlu gömmeden oluşur (85 728 parametre). Model, minGPT'nin kendi örneğindeki sıralama görevinde eğitilmiştir. Kayan nokta model önce matematiksel olarak eşdeğer biçimde dönüştürülmüş, sonra INT8 ağırlık ve aktivasyonlarla, INT16 artık akışla temsil edilmiştir. LayerNorm, softmax ve GELU için yalnızca toplama, çarpma, kaydırma, taban bölme ve taban karekök kullanan tamsayı algoritmalar türetilmiş, her biri için taşma ve doyma sınırları ispatlanmıştır.

**Ana sonuçlar**

- **Doğruluk:** float model, tamsayı model ve RTL, görevin 729 olası girdisinin tamamında %100 doğrudur.
- **Sayısal sadakat:** tamsayı modelin logit SQNR değeri 41,5 dB'dir; float modelle token uyumu %100'dür.
- **Bit-exact doğrulama:** 729 dizi × 11 decode adımı = 8 019 adımın her birinde token, logitler ve 48 artık değer, Python referans modeliyle birebir aynıdır (Verilator 5.052).
- **Performans:** token başına 14 544 çevrim; 100 MHz'de 145 µs/token. Matris-vektör motorunun verimi %98,3'tür.
- **Bellek:** 83 KB INT8 ağırlık ve 3 KB KV önbelleği; harici bellek gerekmez.
- **Hedef kart:** PYNQ-Z2 (Zynq-7020). Çip üzeri öz-test simülasyonda doğrulanmıştır.

**Kapsam notu.** Bu raporun tarihinde Vivado sentezi, zamanlama kapanışı ve kart üzerinde çalıştırma henüz yapılmamıştır. Bunun nedeni Vivado'nun geliştirme bilgisayarı olan macOS'ta çalışmamasıdır. Gerekli bütün scriptler depoda hazırdır. Kaynak kullanımı ile ilgili sayılar bu yüzden hesapla elde edilmiş tahminlerdir.

Rapor iki kısımdan oluşur. Kısım I (Bölüm 1–9) minGPT'nin matematiksel modelini, eğitimini ve çıkarım algoritmasını anlatır. Kısım II (Bölüm 10–21) modelin tamsayı aritmetiğine dönüştürülmesini, donanım mimarisini, performans analizini, doğrulamayı ve kart entegrasyonunu anlatır. Bütün kaynak kod ve üretilen dosyalar `github.com/Muratatceken/mingpt-fpga` adresindedir.

# İçindekiler

**Özet** i

**Kısım I minGPT: Teknik, Algoritmik ve Matematiksel Anlatım** 1

**1 Giriş: dil modelleme ve minGPT** 1

**2 Girdi temsili: token ve konum gömmeleri** 2

**3 Transformer bloğu I: artık akış ve LayerNorm** 3

**4 Transformer bloğu II: nedensel çok başlı öz-dikkat** 4

**5 Transformer bloğu III: MLP ve GELU** 6

**6 Çıkış katmanı, kayıp fonksiyonu ve eğitim** 7

**7 Çıkarım algoritması ve KV önbelleği** 8

**8 Parametre ve işlem sayısı analizi** 9

**9 Deney görevi: sıralama** 10

**Kısım II FPGA İmplementasyonu** 12

**10 Tasarım hedefleri ve kısıtları** 12

**11 Sayı sistemi ve kuantizasyon teorisi** 13
11.1 Düzgün simetrik kuantizasyon . . . 13
11.2 Taşıyıcı biçimler . . . 13
11.3 Yalnızca tamsayı GEMV . . . 14
11.4 Yeniden kuantizasyon (requantization) . . . 14
11.5 Kalibrasyon . . . 15
11.6 Biçim özeti ve sayısal sonuç . . . 15

**12 Matematiksel olarak eşdeğer dönüşümler (katlama)** 16

**13 Tamsayı çekirdekler** 17
13.1 Tamsayı LayerNorm . . . 17
13.2 Tamsayı softmax . . . 18
13.3 GELU: tam tablo . . . 19
13.4 Argmax ve artık toplama . . . 19

**14 Donanım mimarisi** 20
14.1 Üst modül arayüzü . . . 20
14.2 Kontrol FSM'i . . . 20

**15 GEMV motoru** 22
15.1 Dataflow seçimi . . . 22
15.2 Ağırlık ROM düzeni . . . 22

15.3 Boru hattı ve gölge banka . . . . . 23
15.4 Requant birimi . . . . . 24
15.5 Çevrim sayısı . . . . . 24

**16 LayerNorm ve dikkat birimleri** **25**
16.1 Ardışık karekök . . . . . 25
16.2 Ardışık bölücü . . . . . 25
16.3 LayerNorm birimi . . . . . 25
16.4 Dikkat birimi . . . . . 26

**17 Bellek organizasyonu ve FPGA kaynak eşlemesi** **26**

**18 Performans analizi** **27**

**19 Doğrulama metodolojisi** **29**

**20 PYNQ-Z2 entegrasyonu** **30**

**21 Sınırlamalar ve gelecek çalışmalar** **31**

**Kaynakça** **33**

# Şekil Listesi

1 minGPT ileri geçişi . . . 4
2 Nedensel maske ve çok başlı öz-dikkat . . . 5
3 KV önbellekli decode adımı . . . 9
4 Yazılımdan donanıma akış . . . 13
5 Tamsayı GEMV şeridi ve yeniden kuantizasyon . . . 14
6 Float ve tamsayı logitleri . . . 16
7 Tamsayı softmax veri yolu . . . 19
8 GELU ve INT8 GELU tablosu . . . 19
9 `gpt_top` veri yolu . . . 20
10 Kontrol FSM'i . . . 21
11 GEMV motoru . . . 23
12 GEMV boru hattı zamanlaması . . . 24
13 Token başına çevrim profili . . . 28
14 Paralellik derecesinin etkisi . . . 28
15 Referans model hiyerarşisi . . . 29
16 PYNQ-Z2 üst modülü . . . 31

KISIM I

# minGPT: Teknik, Algoritmik ve Matematiksel Anlatım

## 1 Giriş: dil modelleme ve minGPT

Bir dil modeli, sonlu bir sözlük $\mathcal{V}$ üzerindeki token dizilerine olasılık atayan bir fonksiyondur. Olasılığın zincir kuralı, bu ortak dağılımı koşullu dağılımların çarpımına kesin olarak ayırır:

$$p(x_1, x_2, \ldots, x_T) \;=\; \prod_{t=1}^{T} p\left(x_t \mid x_1, \ldots, x_{t-1}\right). \tag{1}$$

Bu ayrışım bir yaklaşım değil, olasılık teorisinin bir özdeşliğidir. Otoregresif bir model her koşullu dağılımı parametreleri $\theta$ olan tek bir sinir ağı $f_\theta$ ile modeller:

$$p_\theta\left(x_t \mid x_{<t}\right) = \operatorname{softmax}\left(f_\theta(x_{<t})\right)_{x_t}, \qquad f_\theta : \mathcal{V}^{t-1} \to \mathbb{R}^{|\mathcal{V}|}. \tag{2}$$

$f_\theta$'nın çıktısı olan $|\mathcal{V}|$ boyutlu vektöre *logit* vektörü denir. Metin üretmek için bu dağılımdan bir token seçilir, diziye eklenir ve işlem tekrarlanır.

**GPT ailesi.** GPT (*Generative Pre-trained Transformer*), $f_\theta$ olarak Vaswani ve ark. [1] tarafından önerilen transformer mimarisinin yalnızca kod çözücü (*decoder-only*) yarısını kullanır. GPT-1 [2], GPT-2 [3] ve GPT-3 [4] aynı temel yapıyı korur. Aralarındaki fark esas olarak katman sayısı, gömme boyutu ve eğitim verisidir. Kod çözücü bloklarda kodlayıcıya yönelik çapraz dikkat bulunmaz; yalnızca nedensel öz-dikkat ve ileri beslemeli ağ vardır.

**minGPT.** minGPT [5], GPT-2 ile aynı mimariye sahip, yaklaşık 300 satırlık bir PyTorch gerçeklemesidir. Depo üç temel dosyadan oluşur:

- `mingpt/model.py`: model tanımı (`GPT`, `Block`, `CausalSelfAttention`, `NewGELU`) ve `generate()` fonksiyonu,

- `mingpt/trainer.py`: genel amaçlı eğitim döngüsü,
- `mingpt/bpe.py`: GPT-2'nin bayt düzeyinde BPE tokenizer'ı.

minGPT eğitim amaçlıdır, ancak mimari olarak eksiksizdir: `GPT.from_pretrained()` ile OpenAI'ın GPT-2 ağırlıklarını yükleyip aynı çıktıları üretebilir. Bu nedenle minGPT üzerinde yapılan bir donanım tasarımı, büyük GPT modellerinin hesap yapısını da temsil eder. Bir token için yapılan işlemler (matris-vektör çarpımı, LayerNorm, softmax, GELU) GPT-2'de de aynıdır; değişen yalnızca boyutlardır.

**Bu çalışmanın kapsamı.** Model GPU/CPU üzerinde float32 ile eğitilir. FPGA üzerinde yalnızca *çıkarım* yapılır: verilen bir önek dizisinden sonraki tokenların üretilmesi. Bu ayrım güncel uygulamalardaki iş bölümüyle uyumludur. Eğitim bir kez ve yüksek hassasiyetle yapılır; çıkarım ise milyonlarca kez, düşük enerji ve gecikmeyle yapılmalıdır.

Rapor boyunca şu notasyon kullanılır: $C$ gömme boyutu (`n_embd` $= 48$), $H$ dikkat başı sayısı ($3$), $d_h = C/H$ baş boyutu ($16$), $L$ katman sayısı ($3$), $V$ sözlük boyutu ($3$) ve $T$ en uzun bağlam (`block_size` $= 11$).

## 2 Girdi temsili: token ve konum gömmeleri

Transformer ham metin değil, tamsayı token kimlikleri alır. Her token, iki tablodan okunan vektörlerin toplamına dönüştürülür.

**Tokenizasyon.** GPT-2 bayt düzeyinde BPE (*Byte Pair Encoding*) kullanır ve sözlüğü 50 257 tokendan oluşur. Bu projedeki sıralama görevinde sözlük yalnızca $\{0, 1, 2\}$ rakamlarıdır ($V = 3$). Her rakam doğrudan kendi token kimliğidir, bu yüzden ayrı bir tokenizer gerekmez.

**Token gömmesi.** Öğrenilebilir bir matris $W_{te} \in \mathbb{R}^{V \times C}$ her token kimliğini $C$ boyutlu bir vektöre eşler. Bu işlem, one-hot kodlanmış bir vektörle matris çarpımına eşdeğer olan bir tablo okumasıdır:

$$e(\tau) = W_{te}[\tau, :] = W_{te}^{\top} \mathbf{1}_{\tau}, \qquad \mathbf{1}_{\tau} \in \{0, 1\}^{V}. \tag{3}$$

Donanım açısından bu eşdeğerlik önemlidir: gömme katmanı hiç çarpıcı gerektirmez, yalnızca bir ROM okumasıdır.

**Konum gömmesi.** Maskesiz öz-dikkat, konum bilgisi verilmezse girdinin sırasına duyarsızdır. Bir permütasyon matrisi $P$ için $\mathrm{Attn}(PX) = P\,\mathrm{Attn}(X)$ olur; dikkat *permütasyona eşdeğişkendir*. Bu durumda model "0 1 2" ile "2 1 0" dizilerini ayırt edemez. Nedensel maske sıra hakkında yalnızca dolaylı ve kısmi bir bilgi verir; bu yüzden açık bir konum kodlaması kullanılır. minGPT, GPT-2 gibi, öğrenilebilir bir konum tablosu $W_{pe} \in \mathbb{R}^{T \times C}$ kullanır. Bu tablo, orijinal transformer'daki sabit sinüzoidal kodlamanın yerine geçer.

**Girdi matrisi.** Uzunluğu $T$ olan bir dizi için birinci bloğa giren matris şudur:

$$X^{(0)} = \begin{bmatrix} W_{te}[\tau_0] + W_{pe}[0] \\ W_{te}[\tau_1] + W_{pe}[1] \\ \vdots \\ W_{te}[\tau_{T-1}] + W_{pe}[T-1] \end{bmatrix} \in \mathbb{R}^{T \times C}. \tag{4}$$

Eğitim sırasında bu toplama `embd_pdrop` $= 0{,}1$ oranında dropout uygulanır; çıkarımda dropout devre dışıdır. Bu projede $W_{te}$ $3 \times 48$, $W_{pe}$ $11 \times 48$ boyutundadır ve iki tablo toplam 672 parametre içerir.

## 3 Transformer bloğu I: artık akış ve LayerNorm

minGPT'nin gövdesi $L$ adet özdeş transformer bloğunun art arda bağlanmasından oluşur. Her blok iki alt katman içerir ve her alt katmanın çıktısı girdisine eklenir (*residual connection*). minGPT, GPT-2 gibi **pre-LN** yapıyı kullanır; LayerNorm alt katmanın girişinde uygulanır:

$$X' = X + \mathrm{Attn}\left(\mathrm{LN}_1(X)\right), \qquad X'' = X' + \mathrm{MLP}\left(\mathrm{LN}_2(X')\right). \tag{5}$$

`model.py`'de bu iki satır `Block.forward()` içinde birebir `x = x + self.attn(self.ln_1(x))` ve `x = x + self.mlpf(self.ln_2(x))` olarak yazılıdır. Son bloğun ardından bir son LayerNorm (`ln_f`) daha uygulanır. Şekil 1 bütün ileri geçişi gösterir.

**Artık akış.** $X$ matrisi bloklar boyunca taşınan bir "ana veri yolu" gibi düşünülebilir. Her alt katman bu yoldan okur, bir düzeltme hesaplar ve sonucu yola geri ekler. $L$ blok sonunda akış $2L$ düzeltmenin toplamını taşır:

$$X^{(L)} = X^{(0)} + \sum_{\ell=1}^{L} \left[ \Delta_\ell^{\mathrm{attn}} + \Delta_\ell^{\mathrm{mlp}} \right]. \tag{6}$$

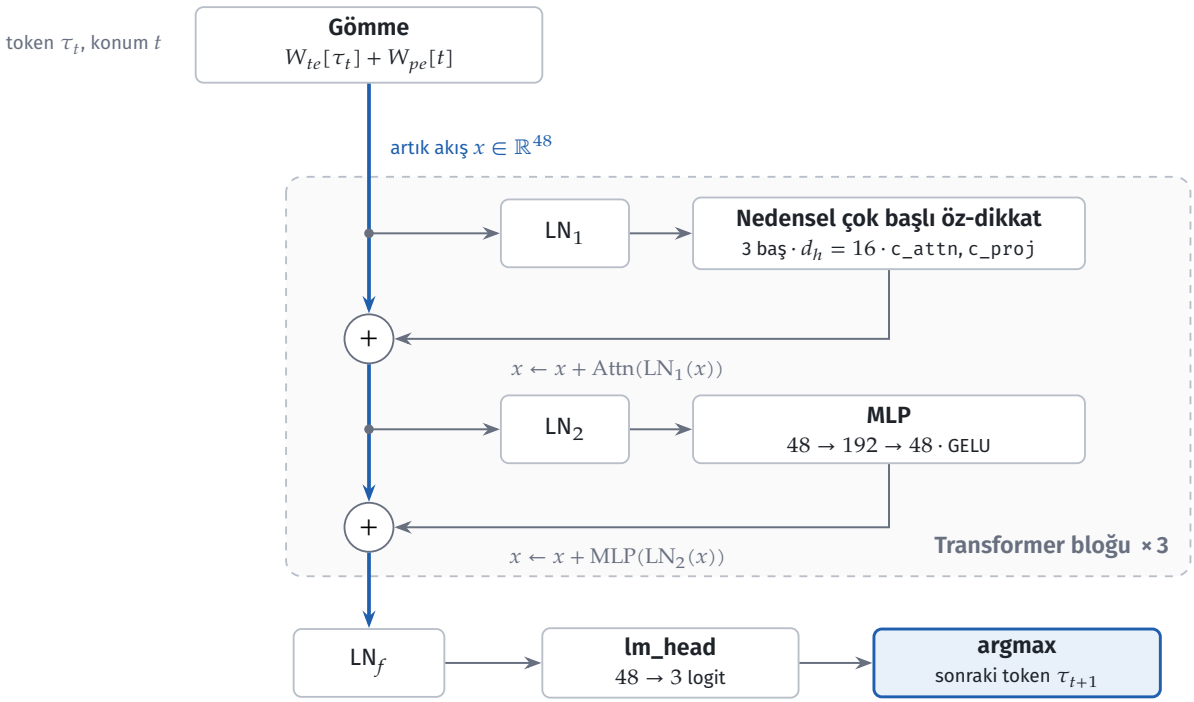
Bu yapının iki sonucu vardır. Birincisi, $\partial X'' / \partial X = I + (\text{alt katman türevleri})$ olduğu için birim matris terimi gradyanın derin ağlarda sönmeden geri yayılmasını sağlar [6]. İkincisi donanımla ilgilidir: akış, ağın en geniş dinamik aralığa sahip tensörüdür, çünkü düzeltmeler birikir. Kısım II'de bu yüzden artık akış INT8 yerine INT16 ile temsil edilecektir.

**Pre-LN ve post-LN.** Orijinal transformer'da LayerNorm toplamadan sonra uygulanır (post-LN). Xiong ve ark. [8], pre-LN yapıda öğrenme oranı ısınması (*warm-up*) olmadan da kararlı eğitim yapılabildiğini göstermiştir. Pre-LN'de artık yol üzerinde hiçbir normalizasyon bulunmaz.

**LayerNorm.** LayerNorm [7] her token vektörünü kendi $C$ özelliği üzerinden normalize eder ve batch boyutundan bağımsızdır:

$$\mu = \frac{1}{C}\sum_{i=1}^{C} x_i, \qquad \sigma^2 = \frac{1}{C}\sum_{i=1}^{C} (x_i - \mu)^2, \qquad \mathrm{LN}(x)_i = \gamma_i \, \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta_i. \tag{7}$$

Burada $\gamma, \beta \in \mathbb{R}^C$ öğrenilebilir parametrelerdir ve $\epsilon = 10^{-5}$'tir. Normalize edilmiş ara vektörü $\hat{z} = (x - \mu)/\sqrt{\sigma^2 + \epsilon}$ ile gösterelim. Kısım II'de kullanılacak üç özellik şunlardır:




**Şekil 1.** minGPT ileri geçişi (`gpt-nano`). Artık akış (mavi) blokların içinden kesintisiz geçer. LayerNorm yalnızca alt katmana giden kolda yer alır (pre-LN). Üç blok art arda uygulanır; ardından $\mathrm{LN}_f$, `lm_head` ve argmax gelir.

- **Ölçek ve kaydırma değişmezliği:** $a > 0$ ve $b$ sabiti için $\hat{z}(ax+b) \approx \hat{z}(x)$ olur ($\epsilon$ ihmal edilince eşitlik tamdır). Girdinin sabit nokta ölçeği bu yüzden normalizasyon sonucunu etkilemez.
- **Küre üzerinde olma:** $\sum_i \hat{z}_i = 0$ ve $\sum_i \hat{z}_i^2 \approx C$; yani $\hat{z}$, yarıçapı $\sqrt{C}$ olan bir kürenin üzerindedir.
- **Sınırlılık:** bu iki koşul her bileşen için $|\hat{z}_i| \le \sqrt{C-1}$ sonucunu verir (Önerme 2). Bu sınır veriden bağımsızdır.

Afin kısım $(\gamma, \beta)$ normalizasyondan sonra geldiği için, Bölüm 12'de bir sonraki doğrusal katmana katlanarak donanımdan tamamen kaldırılacaktır.

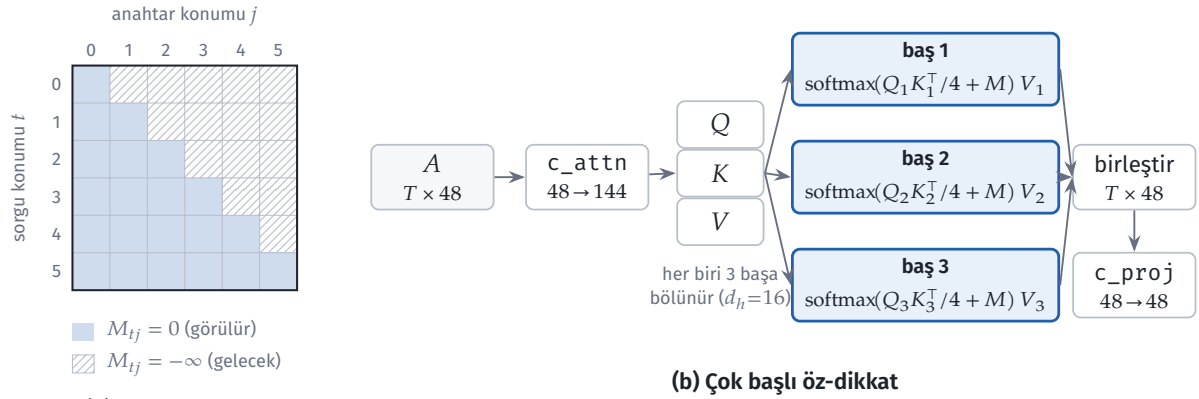
## 4 Transformer bloğu II: nedensel çok başlı öz-dikkat

Öz-dikkat, transformer'da tokenların birbirinden bilgi aldığı tek mekanizmadır. Her token, önceki tokenların değer vektörlerinin ağırlıklı ortalamasını alır. Ağırlıklar, sorgu ile anahtarların benzerliğinden hesaplanır.

**Sorgu, anahtar, değer.** $\mathrm{LN}_1$ çıktısı $A \in \mathbb{R}^{T\times C}$, tek bir doğrusal katmanla (`c_attn`, $C \to 3C$) üç matrise dönüştürülür:

$$[\,Q \mid K \mid V\,] = A\,W_{attn}^{\top} + b_{attn}, \qquad W_{attn} \in \mathbb{R}^{3C\times C}, \quad Q,K,V \in \mathbb{R}^{T\times C}. \tag{8}$$

PyTorch'un `nn.Linear` katmanı $y = xW^{\top} + b$ hesaplar; ağırlık (çıkış × giriş) biçiminde saklanır. Donanımdaki ağırlık ROM'u da bu satır düzenini kullanır.




**(a) Nedensel maske** **(b) Çok başlı öz-dikkat**

**Şekil 2.** (a) $T = 6$ için nedensel maske: $t$ konumu yalnızca $j \le t$ konumlarını görür. (b) Çok başlı öz-dikkat: `c_attn` çıktısı üç başa bölünür; başların çıktıları birleştirilip `c_proj`'dan geçirilir. $1/\sqrt{16} = 1/4$ çarpanı Bölüm 12'de ağırlıklara katlanır.

**Başlara bölme.** $C$ boyutu, her biri $d_h = C/H$ boyutlu $H$ başa bölünür. $h$ numaralı baş $Q$, $K$, $V$'nin $[h\,d_h, (h+1)\,d_h)$ aralığındaki sütunlarını kullanır. Her baş, ölçekli iç çarpım dikkatini bağımsız olarak hesaplar:

$$\text{head}_h = \text{softmax}\left(\frac{Q_h K_h^\top}{\sqrt{d_h}} + M\right) V_h, \qquad M_{tj} = \begin{cases} 0, & j \le t, \\ -\infty, & j > t. \end{cases} \tag{9}$$

Softmax satır bazlıdır: $\text{softmax}(s)_j = e^{s_j} / \sum_k e^{s_k}$. Sonuç negatif olmayan ve toplamı 1 olan bir olasılık vektörüdür. Şekil 2 maskeyi ve çok başlı yapıyı gösterir.

**Neden $\sqrt{d_h}$ ile bölünür?** Sorgu ve anahtar bileşenlerinin bağımsız, sıfır ortalamalı ve birim varyanslı olduğunu varsayalım. Bu durumda iç çarpımın varyansı boyutla doğrusal büyür:

$$\text{Var}(q \cdot k) = \sum_{i=1}^{d_h} \text{Var}(q_i k_i) = \sum_{i=1}^{d_h} \mathbb{E}[q_i^2]\,\mathbb{E}[k_i^2] = d_h. \tag{10}$$

$\sqrt{d_h}$'ye bölmek skorları yeniden birim varyansa getirir. Bölme yapılmasa büyük $d_h$'de skorlar çok büyür, softmax tek bir girdiye doyar ve gradyan sıfıra yaklaşır [1].

**Nedensel maske.** $M$ matrisi, $t$ konumunun yalnızca $j \le t$ konumlarına bakmasını sağlar. $e^{-\infty} = 0$ olduğu için gelecekteki tokenların ağırlığı tam olarak sıfırdır. Bu sayede eğitimde $T$ konumun hepsi paralel hesaplanır, ama her konum yalnızca geçmişi görür.

**Softmax'ın iki özelliği.** Donanım tasarımı için kritik olan iki özellik vardır:

1. **Kaydırma değişmezliği:** her $c$ sabiti için $\text{softmax}(s - c) = \text{softmax}(s)$. $c = \max_j s_j$ seçilirse bütün üsler sıfır veya negatif olur ve $e^x \in (0, 1]$ aralığında kalır; taşma imkânsızdır.
2. **Dışbükey birleşim:** $p_j \ge 0$ ve $\sum p_j = 1$ olduğundan başın çıktısı $y = \sum_j p_j v_j$, değer vektörlerinin dışbükey birleşimidir: her bileşen için $\min_j v_{j,d} \le y_d \le \max_j v_{j,d}$.

**Birleştirme ve çıkış izdüşümü.** Başların çıktıları yan yana konur ve ikinci bir doğrusal katmandan (`c_proj`, $C \to C$) geçirilir:

$$\mathrm{Attn}(A) = [\,\mathrm{head}_1 \mid \cdots \mid \mathrm{head}_H\,]\, W_{proj}^{\top} + b_{proj}. \tag{11}$$

**Tensör boyutları.** $B$ batch boyutu olmak üzere `CausalSelfAttention.forward()` Tablo 1'daki dönüşümleri yapar. Tam bir dizi için skor matrisi $T^2$ elemanlıdır; dikkatin maliyeti katman başına $O(T^2C)$'dir. Bölüm 7'de çıkarımda bu maliyetin token başına $O(tC)$'ye nasıl indiği gösterilecektir.

**Tablo 1.** `CausalSelfAttention.forward()` içindeki tensör dönüşümleri.

| Adım | İşlem | Boyut |
|---|---|---|
| 1 | `c_attn(x).split(C)` | $3 \times (B,T,C)$ |
| 2 | `view(B,T,H,d_h).transpose(1,2)` | $(B,H,T,d_h)$ |
| 3 | `q @ k.transpose(-2,-1) / sqrt(d_h)` | $(B,H,T,T)$ |
| 4 | maske + softmax + dropout | $(B,H,T,T)$ |
| 5 | `att @ v` | $(B,H,T,d_h)$ |
| 6 | `transpose(1,2).view(B,T,C)` | $(B,T,C)$ |
| 7 | `c_proj` + dropout | $(B,T,C)$ |

## 5 Transformer bloğu III: MLP ve GELU

Bloğun ikinci alt katmanı, her tokene ayrı ayrı uygulanan iki katmanlı bir ileri beslemeli ağdır:

$$\mathrm{MLP}(a) = W_2\,\mathrm{GELU}\,(W_1 a + b_1) + b_2, \qquad W_1 \in \mathbb{R}^{4C\times C}, \quad W_2 \in \mathbb{R}^{C\times 4C}. \tag{12}$$

minGPT'de $W_1$ `mlp.c_fc`, $W_2$ ise `mlp.c_proj` katmanıdır. Ara genişlik $4C = 192$'dir. MLP, bloğun $12C^2$ ağırlığının $8C^2$'sini, yani üçte ikisini taşır; hesap yükünün büyük kısmı da buradadır.

**GELU.** Hendrycks ve Gimpel'in [9] önerdiği GELU, girdiyi standart normal dağılımın birikimli dağılım fonksiyonu $\Phi$ ile ağırlıklandırır:

$$\mathrm{GELU}(x) = x\,\Phi(x) = \frac{x}{2}\left(1 + \mathrm{erf}\!\left(\frac{x}{\sqrt{2}}\right)\right). \tag{13}$$

ReLU girdiyi işaretine göre sert biçimde keser. GELU ise bu kesmeyi yumuşak ve türevlenebilir yapar. minGPT, GPT-2 ile aynı sonuçları vermek için `NewGELU` sınıfında tanh yaklaşımını kullanır:

$$\mathrm{GELU}(x) \approx \frac{x}{2}\left(1 + \tanh\!\left(\sqrt{2/\pi}\,\left(x + 0{,}044715\,x^3\right)\right)\right). \tag{14}$$

Bu projede sayısal olarak hesaplanan değerler şunlardır: tanh yaklaşımı ile erf tanımı arasındaki en büyük mutlak fark $4{,}73\times10^{-4}$'tür ($x \approx 2{,}70$). GELU monoton değildir; en küçük değeri yaklaşık $-0{,}170$'tir ve $x \approx -0{,}752$'de alınır.

Donanım açısından erf ve tanh, doğrudan gerçeklenmesi pahalı aşkın fonksiyonlardır. Bölüm 13'te girişin INT8'e kuantize edilmesi sayesinde GELU'nun 256 girişlik bir tabloyla, yaklaşım hatası olmadan gerçeklendiği gösterilecektir (Şekil 8).

**Dropout.** Eğitimde MLP çıktısına `resid_pdrop` $= 0{,}1$ oranında dropout uygulanır. Dropout her elemanı $p$ olasılığıyla sıfırlar, kalanları $1/(1-p)$ ile ölçekler; böylece beklenen değer çıkarımdakiyle aynı kalır. Çıkarımda (`model.eval()`) dropout birim dönüşümdür ve donanımda karşılığı yoktur.

## 6 Çıkış katmanı, kayıp fonksiyonu ve eğitim

Son bloğun çıktısı `ln_f` ile normalize edilir ve bias içermeyen `lm_head` katmanıyla sözlük boyutuna izdüşürülür. minGPT, GPT-2'nin aksine `lm_head` ağırlığını token gömmesiyle paylaşmaz:

$$z_t = W_{head}\,\mathrm{LN}_f\left(x_t^{(L)}\right) \in \mathbb{R}^V, \qquad p_\theta(\cdot \mid x_{\le t}) = \mathrm{softmax}(z_t). \tag{15}$$

**Kayıp fonksiyonu.** Eğitim, hedef tokenların negatif log-olabilirliğini (çapraz entropi) en aza indirir. Hedefi $-1$ olan konumlar kayba katılmaz (`ignore_index=-1`). İşaretli konumların kümesi $\mathcal{M}$ olmak üzere:

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{M}|} \sum_{(b,t)\in\mathcal{M}} \log p_\theta\left(y_{b,t} \mid x_{b,\le t}\right). \tag{16}$$

Çapraz entropinin logitlere göre gradyanı basittir: $\partial\mathcal{L}/\partial z_t = p_t - e_{y_t}$. Rastgele tahmin yapan bir modelin kaybı $\ln V$'dir; $V = 3$ için 1,0986.

**Öğretmen zorlaması.** Eğitimde girdi dizisi $x$ ve bir konum kaydırılmış hedef dizisi $y$ birlikte verilir. Nedensel maske sayesinde $T$ konumun kayıp terimleri tek bir ileri geçişte paralel hesaplanır.

**Optimizasyon: AdamW.** minGPT, Loshchilov ve Hutter'ın [10] ayrık ağırlık azaltmalı Adam yöntemini kullanır. $g_t$ gradyan, $\eta$ öğrenme oranı ve $\lambda$ ağırlık azaltma katsayısı olmak üzere:

$$\begin{aligned} m_t &= \beta_1 m_{t-1} + (1-\beta_1)\,g_t, & v_t &= \beta_2 v_{t-1} + (1-\beta_2)\,g_t^2, \\ \hat{m}_t &= m_t/(1-\beta_1^t), & \hat{v}_t &= v_t/(1-\beta_2^t), \\ \theta_t &= \theta_{t-1} - \eta\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t}+\epsilon} + \lambda\,\theta_{t-1}\right). \end{aligned} \tag{17}$$

**Tablo 2.** minGPT'nin optimizasyon ayarları (`trainer.py`, `model.py::configure_optimizers`).

| Ayar | Değer |
|---|---|
| $\beta_1, \beta_2$ | 0,9 ve 0,95 |
| Ağırlık azaltma $\lambda$ | 0,1, yalnızca `nn.Linear` ağırlıklarına |
| Azaltma uygulanmayanlar | tüm bias'lar, LayerNorm ve gömme ağırlıkları |
| Gradyan kırpma | norm $\le 1{,}0$ (`clip_grad_norm_`) |
| Batch boyutu | 64 |
| Öğrenme oranı | varsayılan $3\times10^{-4}$; bu projede $5\times10^{-4}$ |

**Ağırlık başlatma.** Tüm doğrusal ve gömme ağırlıkları $\mathcal{N}(0, 0{,}02^2)$ dağılımından çekilir. Bias'lar 0, LayerNorm'un $\gamma$'sı 1 ve $\beta$'sı 0 ile başlar. Artık akışa yazan izdüşümlerin (`c_proj`) standart sapması ise $0{,}02/\sqrt{2L}$ alınır. Akış $2L$ düzeltmenin toplamıdır ve bağımsız terimlerin varyansları toplanır; bu ölçekleme akışın varyansını derinlikten bağımsız tutar [3].

## 7 Çıkarım algoritması ve KV önbelleği

Çıkarımda model, verilen bir önek diziyi her adımda bir token uzatır. minGPT'nin `generate()` fonksiyonu bunu her adımda tüm diziyi yeniden işleyerek yapar. Bu çalışmadaki tamsayı model ve donanım ise yalnızca yeni tokenı işler; geçmiş KV önbelleğinde tutulur.

**`generate()` algoritması.** Her adımda şu işlemler yapılır:

1. Bağlam `block_size` uzunluğunu aşarsa son `block_size` token alınır.
2. Bağlamın tamamı için ileri geçiş yapılır ve yalnızca son konumun logitleri $z$ alınır.
3. $z$ sıcaklığa bölünür; istenirse yalnızca en büyük $k$ logit tutulur (*top-k*).
4. $p = \mathrm{softmax}(z)$ hesaplanır; sonraki token örneklenir veya açgözlü olarak $\arg\max_j p_j$ seçilir.
5. Seçilen token diziye eklenir.

Açgözlü kod çözmede softmax gereksizdir. Softmax kesin artan bir fonksiyon olduğundan $\arg\max \mathrm{softmax}(z) = \arg\max z$ sağlanır; donanım bu özelliği kullanır ve çıkışta softmax hesaplamaz.

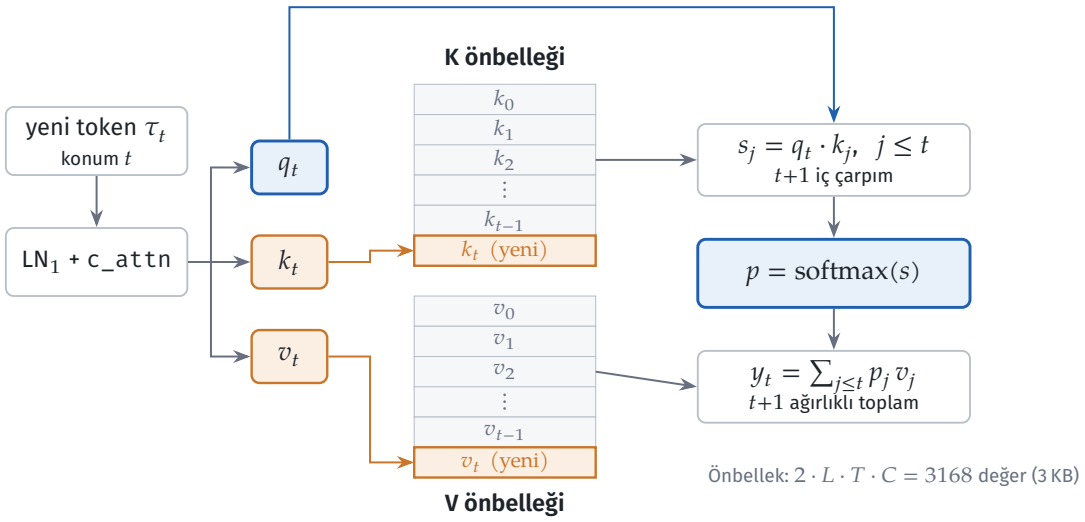
> **Önerme 1** (Nedensel bağımlılık)**.** Nedensel maske altında $j$ konumundaki her ara değer (her katmandaki $x_j, k_j, v_j$) yalnızca $\tau_0, \dots, \tau_j$ tokenlarına bağlıdır.

*İspat.* Katman sırası üzerinden tümevarım. Gömmede $x_j^{(0)}$ yalnızca $\tau_j$ ve $j$'ye bağlıdır. Bir bloğun girişinde iddianın doğru olduğunu varsayalım. LayerNorm, doğrusal katmanlar, GELU ve artık toplama konum bazlıdır; $j$ konumunun çıktısını yalnızca $j$ konumunun girdisinden üretir. Dikkat ise $j$ konumunda yalnızca $i \le j$ konumlarının $k_i, v_i$ değerlerini kullanır ve bunlar varsayım gereği yalnızca $\tau_0, \dots, \tau_i$'ye bağlıdır. Dolayısıyla iddia bloğun çıkışında da doğrudur. □

**Sonuç: KV önbelleği.** Diziye yeni bir token eklendiğinde önceki konumların $k_j$, $v_j$ vektörleri değişmez. Bunlar bir kez hesaplanıp saklanırsa, $t$ adımında yalnızca $t$ konumu işlenir (Şekil 3):

$$
\begin{aligned}
[\, q_t \,|\, k_t \,|\, v_t \,] &= W_{attn}\, \mathrm{LN}_1(x_t) + b_{attn} \qquad (k_t, v_t \text{ önbelleğe yazılır}), \\
y_t^{(h)} &= \sum_{j=0}^{t} \frac{\exp\!\left(q_t^{(h)} \cdot k_j^{(h)} / \sqrt{d_h}\right)}{\sum_{i=0}^{t} \exp\!\left(q_t^{(h)} \cdot k_i^{(h)} / \sqrt{d_h}\right)}\, v_j^{(h)}.
\end{aligned}
\tag{18}
$$

Bu formda bütün doğrusal katmanlar matris-matris çarpımı (GEMM) yerine **matris-vektör çarpımına (GEMV)** dönüşür. Dikkat de $t+1$ uzunluğunda iki iç çarpım dizisine indirgenir.




**Şekil 3.** KV önbellekli bir decode adımı (bir katman, bir baş). Yeni token yalnızca kendi $q_t, k_t, v_t$ vektörlerini üretir; $k_t$ ve $v_t$ önbelleğin $t$ satırına yazılır. Dikkat, önbellekteki $t+1$ satırın hepsini okur.

**Karmaşıklık.** $P_{lin}$ doğrusal katmanların ağırlık sayısı olsun. $N$ token üretmenin çarpma-toplama (MAC) maliyeti Tablo 3'deki gibidir. Önbelleğin bedeli bellektir: $2LTC$ değer. `gpt-nano` için bu $2 \cdot 3 \cdot 11 \cdot 48 = 3168$ değerdir ve INT8 ile 3 KB tutar.

**Tablo 3.** Önbelleksiz ve KV önbellekli üretimin maliyeti.

| **Yöntem** | $t$ **adımının maliyeti** | $N$ **adımın toplamı** |
|---|---|---|
| `generate()` (önbelleksiz) | $(t{+}1)P_{lin} + O((t{+}1)^2 LC)$ | $O(N^2 P_{lin} + N^3 LC)$ |
| KV önbellekli decode | $P_{lin} + 2LC(t{+}1)$ | $O(NP_{lin} + N^2 LC)$ |

Önerme 1 matematiksel bir garantidir. Ek olarak deneysel olarak da doğrulanmıştır: adım adım çalışan tamsayı modelin ürettiği tokenlar, minGPT'nin önbelleksiz `generate()` çıktısıyla 729 girdinin tamamında aynıdır.

## 8 Parametre ve işlem sayısı analizi

Bir minGPT modelinin parametre sayısı ve token başına işlem sayısı yalnızca $(L, C, V, T)$ boyutlarının kapalı bir fonksiyonudur. Bir bloktaki dört doğrusal katman ve iki LayerNorm şu parametreleri taşır:

$$\underbrace{(3C^2 + 3C)}_{\texttt{c_attn}} + \underbrace{(C^2 + C)}_{\texttt{c_proj}} + \underbrace{(4C^2 + 4C)}_{\texttt{c_fc}} + \underbrace{(4C^2 + C)}_{\texttt{mlp.c_proj}} + \underbrace{4C}_{\mathrm{LN}_1, \mathrm{LN}_2} = 12C^2 + 13C. \tag{19}$$

Bloklara gömme tabloları, son LayerNorm ve çıkış katmanı eklenince:

$$N_{param} = L\,(12C^2 + 13C) + (V + T)\,C + 2C + VC. \tag{20}$$

KV önbellekli decode'da her ağırlık bir token için tam bir kez çarpılır; buna $t$ konumundaki dikkatin iç çarpımları eklenir:

$$\mathrm{MAC}(t) = L \cdot 12C^2 + VC + L \cdot 2C\,(t+1), \qquad \mathrm{FLOP} \approx 2 \cdot \mathrm{MAC}. \tag{21}$$

**Tablo 4.** `gpt-nano` ($L = 3$, $C = 48$, $V = 3$, $T = 11$) için parametre ve token başına MAC sayıları.

| Bileşen | Boyut (çıkış × giriş) | Parametre | MAC / token |
|---|---|---|---|
| `c_attn` $(Q, K, V)$ | $144 \times 48$ | 7 056 | 6 912 |
| `c_proj` | $48 \times 48$ | 2 352 | 2 304 |
| `mlp.c_fc` | $192 \times 48$ | 9 408 | 9 216 |
| `mlp.c_proj` | $48 \times 192$ | 9 264 | 9 216 |
| $\mathrm{LN}_1 + \mathrm{LN}_2$ | $2 \times 48$ $(\gamma, \beta)$ | 192 | – |
| **Blok toplamı** | | **28 272** | **27 648** |
| 3 blok | | 84 816 | 82 944 |
| `wte` + `wpe` | $(3 + 11) \times 48$ | 672 | tablo okuması |
| `ln_f` | 48 $(\gamma, \beta)$ | 96 | – |
| `lm_head` | $3 \times 48$ | 144 | 144 |
| Dikkat (3 katman) | | – | 288 $(t + 1)$ |
| **Toplam** | | **85 728** | **83 088 + 288 (t + 1)** |

minGPT başlatılırken "number of parameters: 0.09M" yazar; bu sayı `lm_head`'i saymaz (85 584). Adım başına dikkat payı en fazla 3168/86 256 ≈ %3,7'dir. Hesap yükünün %96'dan fazlası doğrusal katmanlardadır; bu gözlem, donanım mimarisinin merkezine bir GEMV motoru konmasının gerekçesidir.

**Ölçek karşılaştırması.** Aynı formüller GPT-2 small ($L = 12$, $C = 768$, $V = 50\,257$, $T = 1024$) için `lm_head` hariç yaklaşık 124 milyon parametre verir (GPT-2'de `lm_head` token gömmesiyle paylaşılır). Bu model INT8 ile bile yaklaşık 124 MB tutar ve FPGA içindeki SRAM'e sığmaz; harici DRAM ve bant genişliği yönetimi gerektirir. Bu çalışmadaki tasarım ilkeleri aynı kalır; değişen, ağırlıkların nereden ve hangi hızla okunduğudur.

## 9 Deney görevi: sıralama

Model, minGPT'nin `demo.ipynb` örneğindeki sıralama görevinde eğitilmiştir: $\{0, 1, 2\}$ rakamlarından oluşan 6 uzunluğundaki bir diziyi küçükten büyüğe sıralamak. Eğitim sonunda model, olası $3^6 = 729$ girdinin tamamını doğru sıralar.

**Veri biçimi.** Girdi ile çözüm birleştirilir ve modele son token hariç verilir. `0 0 2 1 0 1` girdisi için Tablo 5'taki gibidir. Hedefi $-1$ olan konumlar kayba katılmaz; model yalnızca sıralı çıktıyı öğrenir. Dizi uzunluğu $2 \cdot 6 - 1 = 11$ olduğu için `block_size` = 11'dir. Örneklerin yarısı tekrar eden rakamı çok olan dizilerden seçilir; girdinin hash değerine göre %25'i test kümesine ayrılır.

**Tablo 5.** Sıralama görevinin dizi biçimi (girdi `0 0 2 1 0 1`).

| **Konum** | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Girdi $x$ | 0 | 0 | 2 | 1 | 0 | 1 | 0 | 0 | 0 | 1 | 1 |
| Hedef $y$ | $-1$ | $-1$ | $-1$ | $-1$ | $-1$ | **0** | **0** | **0** | **1** | **1** | **2** |

**Bu görev neden seçildi?**

1. **Kapsamlı doğrulama:** girdi uzayı sonlu ve küçüktür. Doğruluk bir örneklemle tahmin edilmez; bütün girdi uzayı üzerinde kesin olarak ölçülür.
2. **Referanssız kontrol:** bir çıktının doğruluğu modelden bağımsız biçimde kontrol edilebilir: çıktı azalmayan olmalı ve girdiyle aynı histograma sahip olmalıdır. Bu özellik Bölüm 20'deki çip üzeri öz-testin temelidir.
3. **Dikkat gerektirir:** çıktının $k$'inci elemanını üretmek için model girdideki bütün konumlara bakarak her rakamdan kaç tane olduğunu ve kaçının zaten yazıldığını çıkarmalıdır. Bu, tokenlar arası bilgi akışı gerektiren gerçek bir dikkat görevidir.

**Tablo 6.** Model ve eğitim yapılandırması.

| Ayar | Değer |
|---|---|
| Model | `gpt-nano`: $L = 3$, $H = 3$, $C = 48$, $d_h = 16$ |
| Sözlük / bağlam | $V = 3$, $T = 11$ |
| Dropout | 0,1 (gömme, dikkat, artık) |
| İterasyon / batch | 5 000 / 64 |
| Optimizasyon | AdamW, $\eta = 5 \times 10^{-4}$, $\beta = (0{,}9;\ 0{,}95)$, $\lambda = 0{,}1$ |
| Tohum | 3407 |
| Süre | CPU (Apple M serisi) üzerinde yaklaşık 90 s |

**Sonuçlar.** Başlangıç kaybı 1,089'dur ve rastgele tahminin kaybı olan $\ln 3 = 1{,}0986$'ya çok yakındır. Kayıp 500. iterasyonda 0,0104'e, sonraki iterasyonlarda $10^{-3}$ mertebesine iner. Açgözlü kod çözmeyle model 729 girdinin **tamamını** doğru sıralar. Eğitilmiş ağırlıklar depoda `model/gpt_sort.pt` dosyasında saklanır; böylece Kısım II'deki bütün adımlar yeniden eğitim yapılmadan tekrarlanabilir.

KISIM II

# FPGA İmplementasyonu

## 10 Tasarım hedefleri ve kısıtları

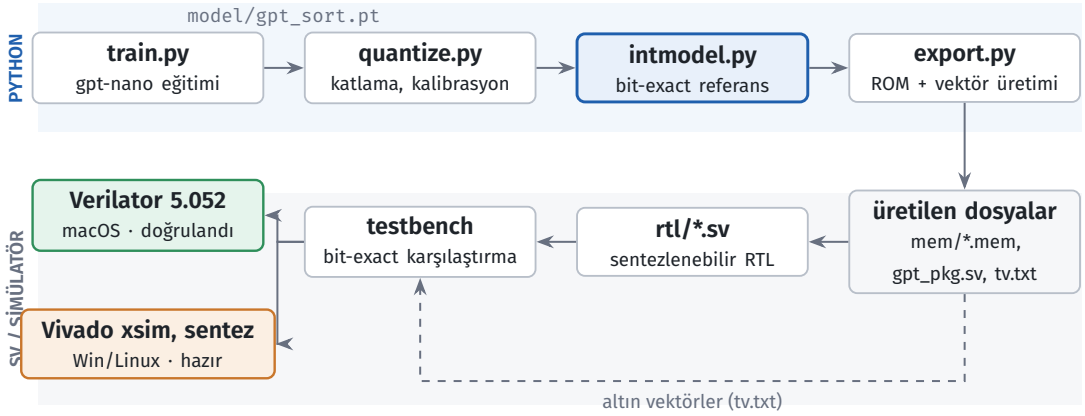
Donanımın amacı, Bölüm 7'deki KV önbellekli decode adımını yalnızca tamsayı aritmetiğiyle ve bütün ağırlıkları çip üzerinde tutarak gerçekleştirmektir. Bu yapılırken bir yazılım referansıyla bit düzeyinde eşdeğerlik korunur.

**Hedefler.**

1. **Yalnızca tamsayı veri yolu:** kayan nokta birimi kullanılmaz; LayerNorm, softmax ve GELU de tamsayı algoritmalarla hesaplanır.
2. **Çip üzeri bellek:** ağırlıklar BRAM'de, ara değerler dağıtık RAM'de ve yazmaçlarda tutulur; DDR ve DMA gerekmez.
3. **Bit-exact doğrulanabilirlik:** her işlem, Python'daki tamsayı referans modelde aynı bit genişliği, yuvarlama ve doyma kurallarıyla tanımlanır (`sw/intmodel.py`). RTL çıktısı bu modelle tolerans olmadan karşılaştırılır.
4. **Parametriklik:** boyutlar ve sabitler, Python tarafından üretilen bir SystemVerilog paketinden (`rtl/gpt_pkg.sv`) okunur. Farklı bir model yeniden eğitilip dışa aktarıldığında RTL değişmez.
5. **Taşınabilir RTL:** kod sentezlenebilir SystemVerilog alt kümesiyle yazılır; hem Verilator hem de Vivado xsim'de çalışır.

**Neden tamsayı?** 7-serisi FPGA'larda bir float32 çarpıcı birden fazla DSP48E1 bloğu, ek LUT ve birkaç çevrimlik gecikme gerektirir. INT8×INT8 çarpımı ise tek bir DSP48E1'in 25 × 18 bitlik çarpıcısının küçük bir kısmını kullanır veya doğrudan LUT'larla yapılabilir. Ağırlık belleği float32'ye göre 4 kat küçülür. Ayrıca tamsayı aritmetik deterministiktir: aynı girdi her platformda aynı bitleri verir. Bit-exact doğrulama ancak bu sayede mümkündür.

**Yazılımdan donanıma akış.** Proje dört Python adımı ve iki simülatör adımından oluşur (Şekil 4). Merkezde, donanımın spesifikasyonu olarak kullanılan tamsayı referans modeli vardır.




**Şekil 4.** Yazılımdan donanıma akış. Python tarafı modeli eğitir, tamsayıya dönüştürür ve ROM içeriklerini, SystemVerilog paketini ve altın test vektörlerini üretir. RTL, Verilator ile bu Mac'te doğrulanmıştır. Vivado adımları hazırdır, ancak henüz çalıştırılmamıştır.

## 11 Sayı sistemi ve kuantizasyon teorisi

Model INT8 ağırlık ve aktivasyonlar, INT32 akümülatörler ve INT16 artık akışla temsil edilir. Gerçel ölçekler çalışma zamanında yalnızca tamsayı çarpma ve kaydırma ile uygulanır. Bu biçim, Jacob ve ark.'nın [11] tamsayı-yalnız çıkarım çerçevesini izler.

### 11.1 Düzgün simetrik kuantizasyon

Gerçel bir $r$ değeri, ölçek $S > 0$ ile $b$ bitlik bir $q$ tamsayısına eşlenir:

$$q = \mathrm{clip}\left(\mathrm{round}\left(\frac{r}{S}\right), -2^{b-1}, 2^{b-1}-1\right), \qquad \hat{r} = S \cdot q. \tag{22}$$

Aralık içindeki her değer için hata $|r - \hat{r}| \le S/2$'dir. Hata $[-S/2, S/2]$ aralığında düzgün dağılımlı kabul edilirse gürültü gücü $S^2/12$ olur. Her ek bit $S$'yi yarıya indirir ve sinyal-kuantizasyon gürültüsü oranını (SQNR) yaklaşık $20\log_{10} 2 \approx 6{,}02\,\mathrm{dB}$ artırır [12].

### 11.2 Taşıyıcı biçimler

- **Ağırlıklar:** INT8, *çıkış kanalı başına* ölçek $S_w[o] = \max_i |W_{oi}|/127$, aralık $[-127, 127]$. Kanal başına ölçek, küçük normlu satırların büyük normlu satırlar yüzünden ezilmesini önler.

- **Aktivasyonlar:** INT8, *tensör başına* sabit ölçek, aralık $[-128, 127]$.
- **Bias:** INT32, ölçeği akümülatörle aynıdır ($S_w[o]\,S_x$); doğrudan akümülatöre eklenebilir.
- **Artık akış:** INT16, ölçeği 2'nin kuvveti olan $S_r = 2^{-FR}$.

## 11.3 Yalnızca tamsayı GEMV

Bir doğrusal katman $y_o = \sum_i W_{oi}x_i + b_o$ için kuantize temsiller yerine yazılır ve ortak çarpan dışarı alınır:

$$y_o \;\approx\; S_w[o]\,S_x \underbrace{\left(\sum_i q^w_{oi}\,q^x_i \;+\; q^b_o\right)}_{acc_o \in \mathbb{Z}}, \qquad q^b_o = \mathrm{round}\left(\frac{b_o}{S_w[o]\,S_x}\right). \tag{23}$$

Parantez içindeki ifade yalnızca tamsayı çarpma ve toplama içerir; donanımın gerçekten hesapladığı değer budur. Çıktının ölçeği $S_y$ ise $q^y_o = \mathrm{round}(M_o\,acc_o)$ olur. Burada $M_o = S_w[o]\,S_x/S_y$ bir gerçel sayıdır.

## 11.4 Yeniden kuantizasyon (requantization)

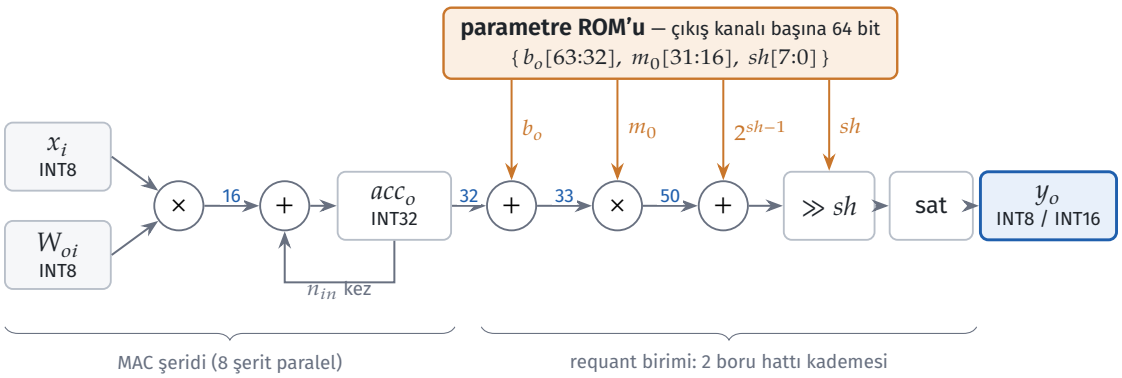
$M_o$, bir tamsayı çarpan ve bir kaydırma ile temsil edilir:

$$M_o \approx m_0 \cdot 2^{-sh}, \qquad 2^{15} \le m_0 < 2^{16}, \qquad \left|\frac{m_0\,2^{-sh} - M_o}{M_o}\right| \le 2^{-16}. \tag{24}$$

$m_0$'ın en yüksek bitinin her zaman 1 olması, 16 bitin tamamının hassasiyete harcanmasını sağlar; bu, kayan noktadaki mantis normalizasyonunun tamsayı karşılığıdır. Donanımda yeniden kuantizasyon şu ifadeyle yapılır:

$$q^y_o = \mathrm{sat}_b\left(\left((acc_o + q^b_o)\cdot m_0 + 2^{sh-1}\right) \gg sh\right). \tag{25}$$

Aritmetik sağa kaydırma taban bölmesidir: $a \gg sh = \lfloor a/2^{sh} \rfloor$. Dolayısıyla $(a + 2^{sh-1}) \gg sh = \lfloor a/2^{sh} + \frac{1}{2} \rfloor$, yani yarıyı yukarı yuvarlamadır. $\mathrm{sat}_b$ sonucu $b$ bitlik işaretli aralığa kırpar. $(b, m_0, sh)$ üçlüsü her çıkış kanalı için parametre ROM'unda 64 bitlik bir kelime olarak saklanır. Veri yolu Şekil 5'te gösterilmiştir.




**Şekil 5.** Tamsayı GEMV şeridi ve yeniden kuantizasyon veri yolu. Mavi sayılar bit genişlikleridir. Parametre ROM'u her çıkış kanalı için bias, $m_0$ ve $sh$ değerlerini sağlar.

**Akümülatör genişliği.** INT8×INT8 çarpımının büyüklüğü en fazla $127 \cdot 128 = 16\,256$'dır. En geniş giriş $N_{in} = 192$ için $|acc| \le 192 \cdot 16\,256 \approx 3{,}1 \cdot 10^6 < 2^{22}$ olur. 32 bitlik akümülatör taşmaz; gereken en az genişlik işaretiyle 23 bittir.

## 11.5 Kalibrasyon

Statik ölçekler, float modelin bir kalibrasyon kümesi üzerinde çalıştırılmasıyla belirlenir: her tensörün en büyük mutlak değeri kaydedilir ve $S = \max|r|/127$ alınır. Bu projede kalibrasyon kümesi görevin **bütün girdi uzayıdır** (729 dizi × 11 konum). Bu nedenle çıkarımda kalibre edilmiş aralığın dışında bir aktivasyon oluşamaz; kırpma hatası sıfırdır.

Artık akışın kalibre edilmiş en büyük değeri 5,936'dır. $FR = \lfloor \log_2(32\,767/5{,}936) \rfloor = 12$ seçilir ve akış **Q3.12** biçiminde temsil edilir (aralık $\pm 8$, çözünürlük $2^{-12}$). Ölçek 2'nin kuvveti olduğundan gömme toplamı ve artık toplama ek bir ölçek dönüşümü gerektirmez. Katman başına kalibre edilen ölçekler Tablo 7'te verilmiştir.

**Tablo 7.** Katman başına kalibre edilen aktivasyon ölçekleri. $S_q$, $1/\sqrt{d_h}$ katlandıktan sonraki sorgu ölçeğidir.

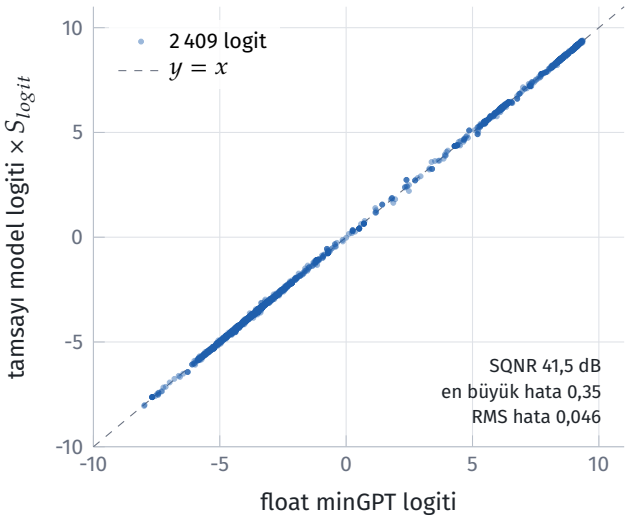
| Katman | $S_q$ | $S_k$ | $S_v$ | $S_h$ (GELU giriş) | $S_g$ (GELU çıkış) |
|---|---|---|---|---|---|
| 0 | 0,004359 | 0,01803 | 0,008157 | 0,02900 | 0,02490 |
| 1 | 0,002940 | 0,008851 | 0,01075 | 0,02246 | 0,01876 |
| 2 | 0,002882 | 0,01326 | 0,01076 | 0,02132 | 0,02125 |

## 11.6 Biçim özeti ve sayısal sonuç

**Tablo 8.** Tensör biçimleri ve seçim gerekçeleri.

| Tensör | Biçim | Ölçek | Gerekçe |
|---|---|---|---|
| Ağırlıklar | INT8 | kanal başına | [11] |
| Bias | INT32 | $S_w S_x$ | akümülatörle aynı ölçek |
| Artık akış | INT16 | $2^{-12}$ (Q3.12) | 2'nin kuvveti; toplama dönüşümsüz |
| LayerNorm çıktısı | INT8 | $2^{-4}$ (Q3.4) | Samuelson sınırı, doyma yok (Önerme 2) |
| $q, k, v$ | INT8 | tensör başına | kalibrasyon |
| Dikkat çıktısı | INT8 | $S_v$ | dışbükeylik, doyma yok (Önerme 4) |
| GELU giriş/çıkışı | INT8 | katman başına | kalibrasyon, tam tablo |
| Softmax ağırlıkları | UQ1.15 | $2^{-15}$ | en büyük terim tam olarak 1 |
| Logitler | INT16 | ortak $S_{logit}$ | argmax için ortak ölçek |

Tamsayı model, float modelle 729 girdinin tamamında aynı tokenları üretir. Öğretmen zorlamalı logit hatasının en büyüğü 0,35, karekök ortalaması 0,046'dır. Logitlerin ortalamadan sapmasına göre ölçülen SQNR 41,5 dB'dir (Şekil 6).




**Şekil 6.** Float minGPT logitleri ile tamsayı modelin logitleri (öğretmen zorlamalı, 146 dizi, her ikinci nokta). Noktalar $y = x$ doğrusu üzerinde toplanır.

## 12 Matematiksel olarak eşdeğer dönüşümler (katlama)

Kuantizasyondan önce, modelin hesapladığı fonksiyonu değiştirmeden donanımı sadeleştiren iki dönüşüm uygulanır. Bu dönüşümler LayerNorm'un afin parametrelerini ve dikkat ölçeğini doğrusal katmanların içine taşır.

**LayerNorm afin katlama.** Her LayerNorm'un ardından bir doğrusal katman gelir. Bu durumda $\gamma$ ve $\beta$, o katmanın ağırlık ve bias'ına katılabilir:

$$W\,(\gamma \odot \hat{z} + \beta) + b \;=\; \underbrace{(W\,\mathrm{diag}(\gamma))}_{W'}\,\hat{z} \;+\; \underbrace{(W\beta + b)}_{b'}. \tag{26}$$

$W'$, $W$'nin $i$'inci sütununun $\gamma_i$ ile çarpılmasıyla elde edilir. `ln_1` `c_attn`'a, `ln_2` `mlp.c_fc`'ye, `ln_f` de `lm_head`'e katlanır. `lm_head`'in bias'ı yoktur; katlama ona $W\beta_f$ bias'ını ekler. Katlamadan sonra donanımdaki LayerNorm birimi yalnızca $\hat{z}$'yi hesaplar. $\gamma$ ve $\beta$ için ne bellek ne çarpıcı gerekir. Bir yan etki olarak $\hat{z}$'nin aralığı artık veriden bağımsız ve matematiksel olarak sınırlıdır. Bu sayede LN çıktısı için kalibrasyon gerektirmeyen sabit bir biçim (Q3.4) kullanılabilir.

**Dikkat ölçeği katlama.** Skor $(q \cdot k)/\sqrt{d_h} = (q/\sqrt{d_h}) \cdot k$ biçiminde yazılabilir. $1/\sqrt{d_h}$ çarpanı, `c_attn`'ın $Q$'yu üreten ilk $C$ satırına katlanır:

$$W'_Q = \frac{W_Q\,\mathrm{diag}(\gamma_1)}{\sqrt{d_h}}, \qquad b'_Q = \frac{W_Q\beta_1 + b_Q}{\sqrt{d_h}}, \qquad \sqrt{d_h} = 4. \tag{27}$$

Bundan sonra donanım skorları doğrudan $q' \cdot k$ olarak hesaplar ve bölme yapmaz.

**Doğrulama.** Katlanmış model float64 ile bağımsız bir NumPy ileri geçişi olarak yazılmıştır. 729 dizinin tümünde her konumdaki logitler, orijinal minGPT çıktısıyla $10^{-4}$ mutlak tolerans içinde aynıdır. Bu kontrol `quantize.py` her çalıştığında otomatik olarak tekrarlanır.

# 13 Tamsayı çekirdekler

Doğrusal olmayan üç işlem kayan nokta ve aşkın fonksiyon kullanmadan, yalnızca toplama, çarpma, kaydırma, karşılaştırma, taban bölme ve taban karekökle hesaplanır. Her algoritma için taşma olmadığı ve doymanın gerekmediği ispatlanmıştır.

## 13.1 Tamsayı LayerNorm

Girdi $N = C = 48$ elemanlı INT16 artık vektörüdür ($S_r = 2^{-12}$). Çıktı Q3.4 biçiminde INT8'dir ($ZF = 4$):

$$\begin{aligned} &s = \textstyle\sum_i x_i, && d_i = Nx_i - s = N\,(x_i - \mu) \quad (\text{tam}),\\ &Q = \textstyle\sum_i d_i^2 + E, && E = \operatorname{round}\left(\epsilon\, N^3/S_r^2\right) \;\Rightarrow\; Q = N^3(\sigma^2+\epsilon)/S_r^2,\\ &rt = \left\lfloor \sqrt{Q} \right\rfloor, && inv = \lfloor K/rt \rfloor, \quad K = \operatorname{round}\left(\sqrt{N}\,2^P\right),\\ &z_i = \operatorname{sat}_8\left(\left(d_i \cdot inv + 2^{P-ZF-1}\right) \gg (P-ZF)\right). \end{aligned} \tag{28}$$

Tanımlar yerine konunca $\hat{z}_i = \sqrt{N}\,d_i/\sqrt{Q}$ olduğu görülür. $inv \approx \sqrt{N}\,2^P/\sqrt{Q}$ olduğundan $d_i \cdot inv/2^{P-ZF} \approx \hat{z}_i\,2^{ZF}$ bulunur; bu, $\hat{z}_i$'nin Q3.4 karşılığıdır. $S_r$ ölçeği yalnızca $\epsilon$ terimi $E$'de görünür; LayerNorm'un ölçek değişmezliği sayesinde başka hiçbir yerde yer almaz. Sabitler: $P = 36$, $E = 18\,554\,259$ ve $K = 476\,102\,500\,705 < 2^{39}$.

> **Önerme 2** (Samuelson eşitsizliği [14])**.** $N$ elemanlı her gerçel vektör için $|x_i - \mu| \le \sigma\sqrt{N-1}$ sağlanır.

*İspat.* $\sum_j (x_j - \mu) = 0$ olduğundan $(x_i - \mu) = -\sum_{j\neq i}(x_j - \mu)$ yazılabilir. Cauchy–Schwarz eşitsizliğiyle

$$(x_i - \mu)^2 \le (N-1)\sum_{j\neq i}(x_j-\mu)^2 = (N-1)\left(N\sigma^2 - (x_i-\mu)^2\right)$$

bulunur; buradan $N(x_i-\mu)^2 \le N(N-1)\,\sigma^2$ elde edilir. □

**Sonuç.** $|\hat{z}_i| \le \sqrt{N-1} = \sqrt{47} \approx 6{,}86$ olur; $\epsilon > 0$ bu değeri yalnızca küçültür. Q3.4'ün en büyük değeri $127/16 = 7{,}94$'tür. LayerNorm çıktısı bu yüzden **hiçbir girdi için doymaz**. Bu garanti kalibrasyondan değil matematikten gelir ve $N \le 64$ olan her model için geçerlidir.

> **Önerme 3** (LayerNorm'da taşma yok)**.** $|d_i \cdot inv| \le K < 2^{39}$.

*İspat.* $|d_i| \le \sqrt{\sum_j d_j^2} \le \sqrt{Q}$ sağlanır. $d_i$ tamsayı olduğu için $|d_i| \le \lfloor\sqrt{Q}\rfloor = rt$ olur. Dolayısıyla $|d_i \cdot inv| \le rt \cdot K/rt = K$. □

23 bitlik $d_i$ ile 39 bitlik $inv$'in çarpımı bu yüzden hiçbir zaman $2^{39}$'u geçmez; 64 bitlik işaretli çarpım güvenlidir. $rt \ge \sqrt{E} \approx 4\,307$ olduğundan karekökün taban almasından gelen bağıl hata en fazla $2{,}3 \cdot 10^{-4}$'tür; çıktıdaki $\pm\frac{1}{2}$ LSB yuvarlamasının yanında ihmal edilebilir.

**Tek geçişli biçim.** $\sum_i d_i^2 = N\,(N\sum_i x_i^2 - s^2)$ özdeşliği tamsayıda *tam* olarak sağlanır. Bu sayede $s$ ve $\sum x^2$ tek geçişte toplanabilir. Kayan noktada bu formun yol açtığı yıkıcı iptal sorunu tamsayı aritmetikte yoktur (gelecek iyileştirme).

## 13.2 Tamsayı softmax

Bir başın $t$ konumundaki skorları $s_j = q \cdot k_j$ ($j = 0..t$) INT32'dir ve ölçekleri $S_s = S_q S_k$'dir. Hesap dört adımda yapılır (Şekil 7).

**(i) Kaydırma.** $m = \max_j s_j$ alınır. Kaydırma değişmezliği sayesinde bütün üsler $\le 0$ olur ve $e_j \in (0, 1]$ aralığında kalır.

**(ii) 2 tabanına geçiş.** $e^x = 2^{x \log_2 e}$ eşitliği kullanılır. Skor farkı, normalize edilmiş bir tamsayı sabitle çarpılarak 2 tabanında sıfır veya pozitif bir üsse dönüştürülür:

$$u_j = (m - s_j) \cdot C_{exp}, \qquad C_{exp}\, 2^{-F} \approx S_q S_k \log_2 e, \qquad 2^{15} \le C_{exp} < 2^{16}. \tag{29}$$

Katman başına $(C_{exp}, F)$ değerleri $(60\,882;\ 29)$, $(40\,315;\ 30)$ ve $(59\,216;\ 30)$'dur. Bu durumda $e_j = 2^{-u_j / 2^F}$ olur.

**(iii) Aralık indirgeme ve tablo.** $u_j$, tam kısım $n$ ve kesir kısmı olarak ayrılır. Tam kısım bir sağa kaydırmaya, kesrin üst 8 biti 256 girişlik bir tabloya dönüşür:

$$n = u_j \gg F, \qquad k = (u_j \gg (F-8)) \,\&\, 255, \qquad \mathrm{LUT}[k] = \mathrm{round}\left(2^{15} \cdot 2^{-k/256}\right),$$
$$e_j = \begin{cases} \mathrm{LUT}[k] \gg n, & n \le 15, \\ 0, & n > 15. \end{cases} \tag{30}$$

$e_j$ UQ1.15 biçimindedir ve en büyük skor için tam olarak $2^{15}$ $(= 1{,}0)$ değerini alır.

**Hata analizi.** Kesrin üst 8 bite kesilmesi üste $2^{-8}$'den küçük bir hata ekler. Bu, $e_j$'de $2^{2^{-8}} - 1 \approx$ %0,27'den küçük bir bağıl hata demektir. Bu hata bütün terimleri aynı yönde etkilediği için $E$'ye bölünürken kısmen birbirini götürür. $\gg n$ kaydırmasının kesmesi en fazla $2^{-15}$ mutlak hata ekler. $n > 15$ için sıfır alınan terimler en büyük terimin $2^{-16}$'sından küçüktür.

**(iv) Ertelenmiş normalizasyon.** Her olasılığı ayrı ayrı bölmek yerine önce ağırlıklı toplam alınır; bölme sonda tek bir karşılık (reciprocal) ile yapılır:

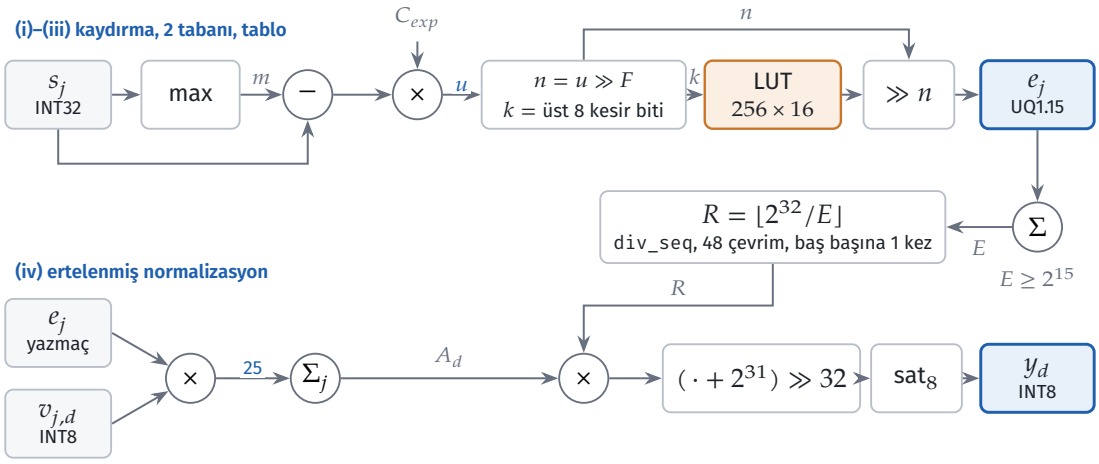
$$A_d = \sum_{j=0}^{t} e_j\, v_{j,d}, \qquad E = \sum_{j=0}^{t} e_j, \qquad R = \left\lfloor \frac{2^{32}}{E} \right\rfloor, \qquad y_d = \mathrm{sat}_8\left(\left(A_d R + 2^{31}\right) \gg 32\right). \tag{31}$$

Böylece baş başına **tek bir bölme** yapılır. En büyük terim için $e = 2^{15}$ olduğundan $E \ge 2^{15}$ olur; sıfıra bölme imkânsızdır ve $R \le 2^{17}$'dir.

> **Önerme 4** (Dışbükeylik ve doymazlık)**.** Dikkat çıktısı $y_d$, ölçeği $S_y = S_v$ seçildiğinde hiçbir girdi için doymaz.

*İspat.* $p_j = e_j / E \ge 0$ ve $\sum_j p_j = 1$ olduğundan $y_d$, $v_{j,d}$ değerlerinin dışbükey birleşimidir; $\min_j v_{j,d} \le y_d \le \max_j v_{j,d}$ sağlanır. Tamsayı tarafta $R$ taban alınarak küçüldüğü için $|A_d R / 2^{32}| \le |A_d| / E \le 127$ olur. □

Bu yöntem, I-BERT'in [13] polinom tabanlı `i-exp` fonksiyonuna bir alternatiftir. FPGA'da küçük LUT-ROM'lar ucuz olduğu için tablo tercih edilmiştir.



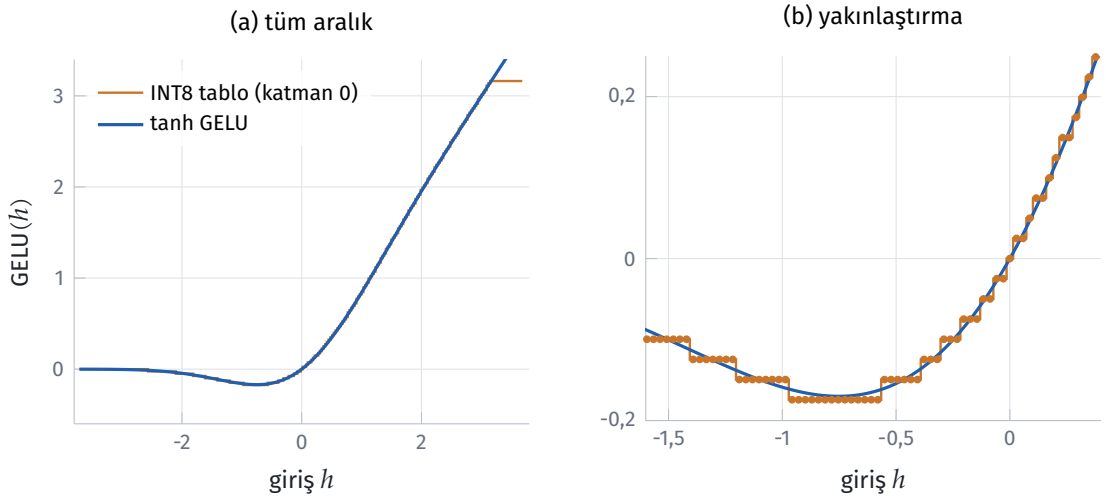

**Şekil 7.** Tamsayı softmax ve ertelenmiş normalizasyon. Üstel ağırlıklar bir çarpma, bir tablo ve bir kaydırmayla hesaplanır. Baş başına tek bir bölme yapılır.

## 13.3 GELU: tam tablo

`c_fc` çıktısı INT8'e $(S_h)$ yeniden kuantize edilir. Bu yüzden GELU'nun girdisi yalnızca 256 farklı değer alabilir ve fonksiyon bir tabloyla *tam olarak* temsil edilir:

$$\mathrm{GELU_LUT}_\ell[q] = \mathrm{sat}_8\left(\mathrm{round}\left(\mathrm{GELU}(q \cdot S_h)/S_g\right)\right), \qquad q \in \{-128, \ldots, 127\}. \tag{32}$$

Tablo, minGPT'nin tanh tabanlı GELU'sunu birebir kullanır. Hata yalnızca giriş ve çıkış kuantizasyonundan gelir; bir fonksiyon yaklaşımı yoktur (Şekil 8). Her katmanın kendi tablosu vardır $(3 \times 256$ bayt$)$. Tablo, INT8 değerin iki tümleyenli bitleriyle doğrudan adreslenir: adres $= \{\ell, h[7{:}0]\}$.




**Şekil 8.** Katman 0'ın gerçek GELU tablosu (turuncu) ve minGPT'nin tanh GELU'su (mavi). (b) Yakınlaştırmada INT8 adımları görülür. (a)'nın sağ ucundaki düz kısım çıkış doymasıdır: $S_g$ en büyük *gözlenen* çıkıştan kalibre edildiği için yalnızca kalibrasyonda hiç görülmeyen pozitif girişlerde oluşur.

## 13.4 Argmax ve artık toplama

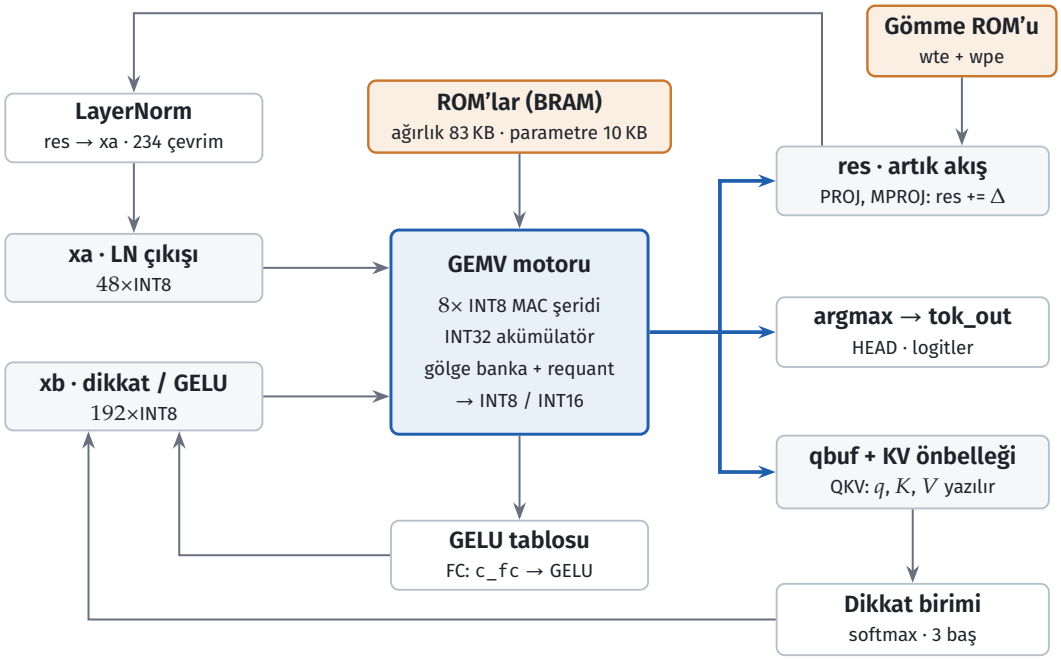
`lm_head` çıktıları ortak bir ölçeğe $(S_{logit}$, INT16$)$ yeniden kuantize edilir. Kanal başına ölçek farkları yeniden kuantizasyonda yok olur; tamsayı karşılaştırma bu yüzden doğrudur. Eşitlik

durumunda en küçük indeks seçilir ve referans model de aynı kuralı uygular. GEMV motoru 8'in katı çıkış istediği için `lm_head` 8 satıra sıfır ağırlıklarla doldurulur; fazla satırlar argmax'a katılmaz.

Artık toplama, `c_proj` ve `mlp.c_proj` çıktılarını doğrudan $S_r$ ölçeğinde INT16'ya yeniden kuantize eder ve $r \leftarrow \mathrm{sat}_{16}(r+\Delta)$ uygular. Gömme adımı da aynı biçimdedir: $x^{(0)} = \mathrm{sat}_{16}(W_{te}[\tau] + W_{pe}[t])$.

## 14 Donanım mimarisi

Hızlandırıcı, bir token için minGPT'nin bütün ileri geçişini katman katman ve sırayla yürütür. Bütün doğrusal katmanlar aynı GEMV motorunu paylaşır; LayerNorm ve dikkatin kendi ardışık birimleri vardır (Şekil 9). GEMV motoru girdisini LN tamponundan (xa) veya dikkat-/GELU tamponundan (xb) alır. Çıktısını o anki duruma göre dört hedeften birine yazar. Artık akış (res) ve KV önbelleği, bloklar ve tokenlar arasında durumu taşıyan iki bellektir.




**Şekil 9.** `gpt_top` veri yolu. Turuncu kutular ROM'lardır, gri kutular tamponlardır. Mavi oklar GEMV çıkış akışının olası hedefleridir.

### 14.1 Üst modül arayüzü

Bir dizi üretmek için `pos` $= 0, 1, 2, \ldots$ sırayla verilir. KV önbelleği konumlar arasında korunur.

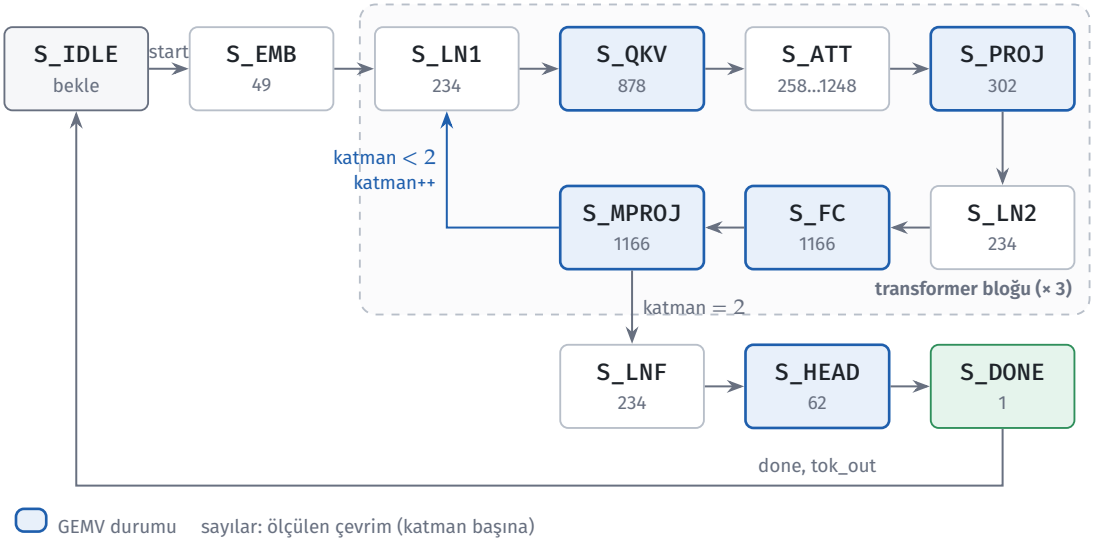
### 14.2 Kontrol FSM'i

FSM, `model.py`'deki `forward()` fonksiyonunu satır satır izler (Şekil 10). Her durumun ilk çevriminde (`kick`) ilgili birim başlatılır; durum, birimin `done` darbesini bekler. `S_LN1`'den

**Tablo 9.** `rtl/gpt_top.sv` portları.

| Port | Yön | Açıklama |
|---|---|---|
| `clk`, `rst_n` | giriş | saat, senkron aktif-düşük reset |
| `start` | giriş | 1 çevrimlik darbe: `tok_in` tokenını `pos` konumunda işle |
| `tok_in`, `pos` | giriş | token (2 bit) ve konum (4 bit, $0 \le$ `pos` $< 11$) |
| `busy`, `done` | çıkış | meşgul / bitti (1 çevrimlik darbe) |
| `tok_out` | çıkış | en büyük logitin indeksi, yani sonraki token |
| `logits` | çıkış | 3×INT16 logit |
| `dbg_addr`, `dbg_res` | giriş/çıkış | artık tamponunun asenkron okunması (test için) |

`S_MPROJ`'a kadar olan yedi durum her katman için tekrarlanır. GEMV'nin ağırlık taban adresi katman sayacından hesaplanır: `layer`·`W_PER_LAYER`+`W_OFF_*`.




**Şekil 10.** Kontrol FSM'i. Mavi durumlar GEMV motorunu kullanır. Sayılar katman başına ölçülen çevrimlerdir; dikkat süresi konuma ($t$) bağlıdır.

**Tasarım kararları.**

- **Zaman çoklamalı birimler:** tek GEMV motoru ve tek LN birimi bütün katmanlar için sırayla kullanılır; alan küçük tutulur.
- **Akış hâlinde çıktı:** GEMV çıktısı (indeks, değer) çiftleri olarak akar. Hedef tampon FSM durumundan seçildiği için ara tampon gerekmez.
- **Kaynaştırılmış GELU:** GELU tablosu FC çıktısının yazma yoluna yerleştirilmiştir; ayrı bir aşama yoktur.
- **Akış hâlinde argmax:** HEAD çıktıları akarken en büyük logit güncellenir; softmax hesaplanmaz.

**Tablo 10.** FSM durumları ve `model.py` karşılıkları.

| Durum | Birim | Okur → yazar | `model.py` |
|---|---|---|---|
| `S_EMB` | gömme ROM'u | wte, wpe → res | `wte(idx) + wpe(pos)` |
| `S_LN1` | layernorm | res → xa | `ln_1(x)` |
| `S_QKV` | gemv 48 → 144 | xa → qbuf, K, V | `c_attn` |
| `S_ATT` | attention | qbuf, K, V → xb | `softmax(q k`$^{\top}$`) v` |
| `S_PROJ` | gemv 48 → 48 | xb → res (+=) | `x + c_proj(y)` |
| `S_LN2` | layernorm | res → xa | `ln_2(x)` |
| `S_FC` | gemv 48 → 192 + GELU | xa → xb | `gelu(c_fc(·))` |
| `S_MPROJ` | gemv 192 → 48 | xb → res (+=) | `x + mlp.c_proj(·)` |
| `S_LNF` | layernorm | res → xa | `ln_f(x)` |
| `S_HEAD` | gemv 48 → 8 + argmax | xa → tok_out | `lm_head` + argmax |

# 15 GEMV motoru

Token başına hesabın %96'sından fazlası matris-vektör çarpımı olduğu için tasarımın merkezinde $P = 8$ şeritli, **çıkış-durgun** (*output-stationary*) bir GEMV motoru vardır (Şekil 11). Motor, ölçülen 10 598 çevrimin 10 416'sında faydalı iş yapar; verimi %98,3'tür.

## 15.1 Dataflow seçimi

Sze ve ark. [15] hızlandırıcıları hangi verinin işlem biriminde sabit kaldığına göre sınıflandırır: ağırlık-durgun, girdi-durgun ve çıkış-durgun. Decode sırasındaki GEMV için çıkış-durgun yapının gerekçesi şudur:

- **Ağırlıklar yeniden kullanılmaz:** her ağırlık bir token için tam bir kez çarpılır; ağırlıkları işlem biriminde tutmak bir şey kazandırmaz.
- **Girdi $P$ kez yeniden kullanılır:** aynı $x_i$, $P$ farklı çıkış nöronu tarafından kullanılır; bütün şeritlere yayınlanır.
- **Kısmi toplamlar yerinde kalır:** her şeridin INT32 akümülatörü $n_{in}$ çevrim boyunca kendi yazmacında birikir.

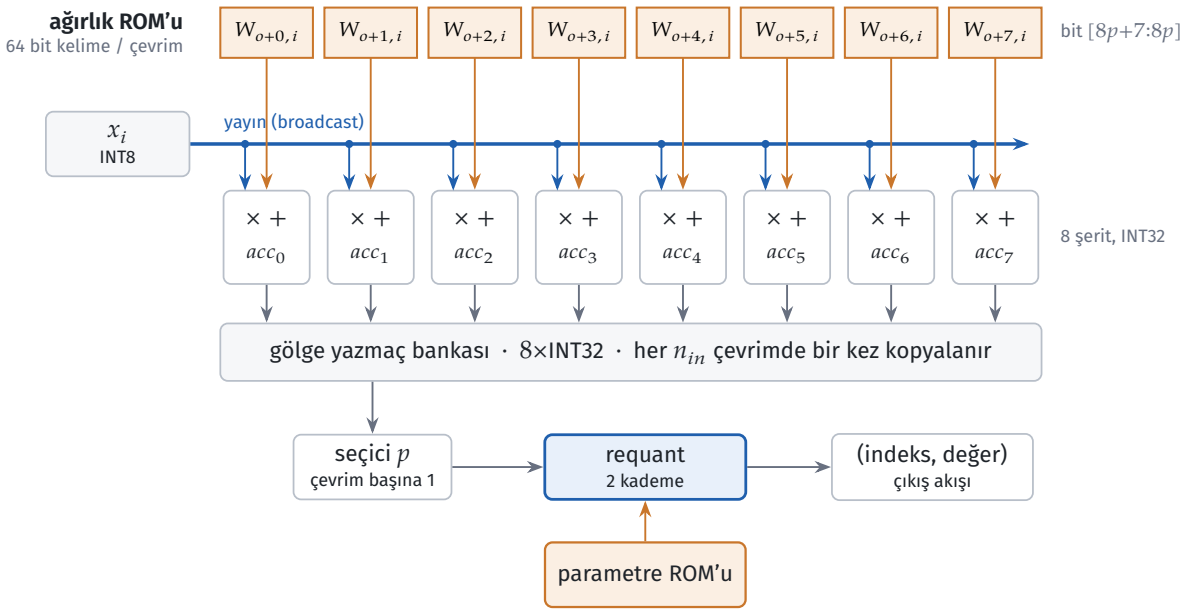
## 15.2 Ağırlık ROM düzeni

ROM kelimesi, $P = 8$ çıkışın aynı sütundaki ağırlıklarını taşır; şerit $p$ kelimenin $[8p{+}7 : 8p]$ bitlerini kullanır:

$$\text{word}(o_b, i) = \{\, W_{o_b P+7,\, i},\ \ldots,\ W_{o_b P,\, i} \,\}, \qquad acc_p \mathrel{+}= W_{o_b P+p,\, i} \cdot x_i \quad (p = 0..7). \tag{33}$$

Her doğrusal katman için önce çıkış bloğu $o_b$, sonra giriş indeksi $i$ sırasıyla yazılır. Adres bu yüzden çağrı boyunca birer birer artan bir sayaçtır:

$$\text{addr} = \ell \cdot 3456 + \text{OFF}_{lin} + o_b \cdot n_{in} + i, \qquad \text{OFF} = \{\texttt{attn}{:}\,0,\ \texttt{proj}{:}\,864,\ \texttt{fc}{:}\,1152,\ \texttt{mproj}{:}\,2304\}. \tag{34}$$




**Şekil 11.** GEMV motoru. Her çevrimde tek bir $x_i$ sekiz şeride yayınlanır ve ağırlık ROM'undan sekiz çıkışın aynı sütundaki ağırlıklarını taşıyan 64 bitlik tek bir kelime okunur. Akümülatörler bir gölge bankaya kopyalanır ve tek bir requant birimine sırayla aktarılır.

`lm_head` 10 368 adresinden başlar; ROM'un toplam derinliği 10 416 kelimedir (83 328 bayt). Parametre ROM'u aynı sırayı çıkış kanalı başına bir kelimeyle izler (1 304 kelime). MAC döngüsünün RTL'deki karşılığı şudur:

```
always_comb begin
  for (int p = 0; p < P; p++) begin
    prod[p]   = $signed(w_word[8*p +: 8]) * x1;        // INT8 x INT8 -> INT16
    acc_nx[p] = (first1 ? 32'sd0 : acc[p]) + prod[p];  // INT32 birikim
  end
end
```

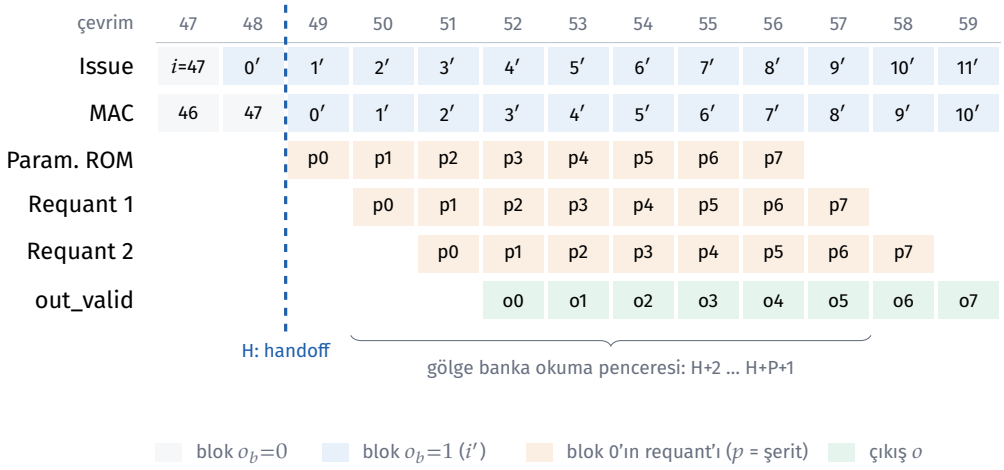
Bloğun ilk elemanında akümülatör sıfır yerine doğrudan çarpımla yüklenir (`first1`); ayrı bir temizleme çevrimi gerekmez.

## 15.3 Boru hattı ve gölge banka

Motor üç aşamalıdır: *issue* (ROM ve aktivasyon adresi), *MAC* (ROM verisi geçerli, 8 şerit biriktirir) ve *requant*. Bloğun son elemanında 8 akümülatör gölge bankaya kopyalanır (*handoff*). Gölge banka her çevrimde bir değer olmak üzere requant birimine aktarılır; bu sırada şeritler bir sonraki 8 çıkışı hesaplamaya başlamıştır. Şekil 12 zamanlamayı gösterir.

> **Önerme 5** (Gölge banka güvenliği)**.** $n_{in} \geq P + 1$ ise gölge bankadaki hiçbir değerin üzerine okunmadan yazılmaz.

*İspat.* H çevriminde yazılan gölge banka H+2 ...H+$P$ + 1 çevrimlerinde okunur. Bir sonraki handoff H+$n_{in}$ çevriminin sonundadır. H+$P$ + 1 ≤ H+$n_{in}$ koşulu $n_{in} \geq P + 1$'e denktir. Modeldeki en küçük $n_{in}$ 48'dir; RTL koşulu simülasyonda bir `assert` ile denetler. □




**Şekil 12.** GEMV boru hattı zamanlaması ($n_{in} = 48$, $P = 8$). Blok 0'ın son MAC'i 48. çevrimdedir; akümülatörler bu çevrimin sonunda gölge bankaya kopyalanır (H). Blok 1'in MAC'leri boşluksuz devam ederken blok 0'ın sekiz çıkışı 52–59. çevrimlerde akar.

## 15.4 Requant birimi

Bölüm 11'teki ifade iki boru hattı aşamasında hesaplanır: önce 33 bitlik $(acc+b)$ toplamı işaretli 17 bitlik $\{0, m_0\}$ ile çarpılır ve 64 bitlik yazmaca alınır; sonra $2^{sh-1}$ eklenir, aritmetik kaydırma yapılır ve sonuç `out16` bayrağına göre INT8 veya INT16 aralığına doyurulur:

```
always_ff @(posedge clk) prod_r <= sum * m0_s;              // kademe 1
assign shifted = (prod_r + (64'sd1 <<< (sh_r - 6'd1))) >>> sh_r;
always_ff @(posedge clk) out <= sat(shifted, out16_r);      // kademe 2
```

## 15.5 Çevrim sayısı

Bir GEMV çağrısı $(n_{out}/P) \cdot n_{in} + 14$ çevrim sürer. Ek 14 çevrim son bloğun yeniden kuantizasyonundan, boru hattının dolup boşalmasından ve FSM geçişinden gelir.

**Tablo 11.** GEMV çağrılarının ideal ve ölçülen çevrim sayıları.

| Katman | $n_{out} \times n_{in}$ | İdeal | Ölçülen |
|---|---|---|---|
| `c_attn` | $144 \times 48$ | 864 | 878 |
| `c_proj` | $48 \times 48$ | 288 | 302 |
| `mlp.c_fc` | $192 \times 48$ | 1 152 | 1 166 |
| `mlp.c_proj` | $48 \times 192$ | 1 152 | 1 166 |
| `lm_head` | $8 \times 48$ | 48 | 62 |

Tüm token için ideal toplam $3 \cdot 3456 + 48 = 10\,416$ çevrimdir; bu, ağırlık ROM'unun derinliğine eşittir, çünkü motor her çevrimde bir ROM kelimesi tüketir.

# 16 LayerNorm ve dikkat birimleri

LayerNorm ve dikkat hesap yükünün küçük bir kısmını oluşturur. Bu yüzden ikisi de çevrim başına bir işlem yapan ardışık birimler olarak tasarlanmıştır: LN başına 234 çevrim, dikkat için katman başına ortalama 752 çevrim.

## 16.1 Ardışık karekök

Taban karekök, elle karekök almaya benzeyen *basamak basamak* (*digit-by-digit*) yöntemle hesaplanır. Her çevrimde girdiden 2 bit tüketilir ve köke 1 bit eklenir; 64 bitlik girdi için 32 çevrim sürer:

$$rem \leftarrow 4\,rem + (\text{sıradaki 2 bit}), \qquad trial = 4\,root + 1,$$
$$(rem,\ root) \leftarrow \begin{cases} (rem - trial,\ 2\,root + 1), & rem \geq trial, \\ (rem,\ 2\,root), & \text{aksi hâlde.} \end{cases} \tag{35}$$

**Değişmez:** $i$ adımdan sonra $N_i$ girdinin en üst $2i$ biti olsun. $root_i = \lfloor \sqrt{N_i} \rfloor$ ve $rem_i = N_i - root_i^2 \leq 2\,root_i$ sağlanır. Aday kök $2r + 1$'dir ve $(2r+1)^2 = 4r^2 + 4r + 1$ olduğundan karşılaştırılacak değer tam olarak $4r + 1$'dir.

## 16.2 Ardışık bölücü

Taban bölme, işaretsiz *geri yüklemeli* (*restoring*) yöntemle her çevrimde bir bölüm biti üretir (48 bit için 48 çevrim): $r \leftarrow 2r + \text{bit}$; $r \geq D$ ise $r \leftarrow r - D$ ve bölüm biti 1. Değişmez: $q_i = \lfloor N_i / D \rfloor$, $0 \leq r_i < D$. Bölücü LayerNorm'da $K/rt$, softmax'ta $2^{32}/E$ için kullanılır ve her ikisinde de yalnızca *bir kez* çalışır. Her iki birim, birim testinde 2 000 rastgele girdiyle referans gerektirmeyen koşullara karşı doğrulanmıştır: $r^2 \leq n < (r+1)^2$ ve $q\,d \leq n < (q+1)\,d$.

## 16.3 LayerNorm birimi

**Tablo 12.** LayerNorm biriminin aşamaları (toplam 234 çevrim, ölçülen).

| Aşama | İşlem | Çevrim |
|---|---|---|
| SUM | $s = \sum x_i$ | 48 |
| VAR | $d_i = 48x_i - s$ (yazmaçlanır); $Q \mathrel{+}= d_i^2$ | $48 + 1$ |
| SQRT | $rt = \lfloor \sqrt{Q + E} \rfloor$ | $32 + 1$ |
| DIV | $inv = \lfloor K/rt \rfloor$ | $48 + 1$ |
| OUT | $z_i = \mathrm{sat}_8((d_i\,inv + 2^{31}) \gg 32)$ | $48 + 2$ |

$d_i$ iki kez hesaplanır (VAR ve OUT); böylece 48 elemanlı bir $d$ tamponuna gerek kalmaz. Bit genişlikleri ihtiyacın üstünde seçilmiştir: $d$ 32 bit (gereken 23), $Q$ 64 bit (gereken yaklaşık 50), $inv$ 48 bit, $d \cdot inv$ 64 bit (Önerme 3'e göre $2^{39}$ ile sınırlı).

### 16.4 Dikkat birimi

Dikkat birimi, $t$ konumundaki tek sorgu için üç başı sırayla işler. Önbellek adreslemesi doğrusaldır:

$$\mathrm{addr}_{KV}(\ell, j, h, d) = (\ell\, T + j)\, C + h\, d_h + d, \qquad 0 \le \mathrm{addr}_{KV} < L\, T\, C = 1584. \tag{36}$$

QKV GEMV'si $(\ell, t)$ satırını yazar, dikkat $(\ell, 0..t)$ satırlarını okur. Yeni bir diziye $t = 0$ ile başlanırsa eski satırlar silinmez; $j > t$ satırları hiç okunmadığı için bu zararsızdır.

**Tablo 13.** Dikkat biriminin baş başına aşamaları.

| Aşama | İşlem | Çevrim |
|---|---|---|
| `SCORE` | $s_j = \sum_d q_d\, k_{j,d}$ (INT8×INT8), koşan maksimum $m$ | $16(t+1)$ |
| `EXP` | kademe 1: $u_j = (m - s_j)\, C_{exp}$; kademe 2: $e_j = \mathrm{LUT}[k] \gg n$, $E \mathrel{+}= e_j$ | $(t+1)+2$ |
| `DIV` | $R = \lfloor 2^{32}/E \rfloor$ | $48+2$ |
| `AV` | $A_d = \sum_j e_j\, v_{j,d}$ (UQ1.15×INT8); $y_d = \mathrm{sat}_8((A_d R + 2^{31}) \gg 32)$ | $16(t+1)+1$ |

Skorlar ($11 \times 32$ bit) ve üstel ağırlıklar ($11 \times 16$ bit) birimin içindeki yazmaçlarda tutulur. AV aşamasında dış döngü $d$, iç döngü $j$ üzerindedir; böylece her $y_d$, $j$ döngüsü biter bitmez yazılabilir. Bir katmanın dikkat süresi yaklaşık 3 $(33\,(t+1)+53)$ çevrimdir; $t = 0$'da 258, $t = 10$'da 1 248 çevrim.

## 17 Bellek organizasyonu ve FPGA kaynak eşlemesi

Tasarımın bellek gereksinimi yaklaşık 95 KB ROM ve 4 KB RAM'dir. Büyük ROM'lar blok RAM'e (BRAM), küçük tamponlar ve tablolar dağıtık RAM'e (LUTRAM) veya yazmaçlara eşlenir. Bu bölümdeki kaynak sayıları hesapla elde edilmiş **tahminlerdir**; kesin değerler Vivado sentez raporundan alınacaktır.

**Tablo 14.** Bellekler ve beklenen gerçeklemeleri.

| Bellek | Boyut | Okuma | Beklenen gerçekleme |
|---|---|---|---|
| Ağırlık ROM'u | 10 416 × 64 bit (83,3 KB) | senkron | ≈ 21 RAMB36 (512 × 72 düzeni) |
| Parametre ROM'u | 1 304 × 64 bit (10,4 KB) | senkron | ≈ 3 RAMB36 |
| `wte` / `wpe` | 144 / 528 × 16 bit | senkron | LUT-ROM veya RAMB18 |
| GELU tablosu | 768 × 8 bit | asenkron | LUT-ROM |
| exp tablosu | 256 × 16 bit | asenkron | LUT-ROM |
| K ve V önbelleği | 2 × 1 584 × 8 bit | asenkron | LUTRAM |
| Artık tampon | 48 × 16 bit | asenkron, 3 port | yazmaç |
| xa, xb, qbuf | 48, 192, 48 × 8 bit | asenkron | LUTRAM |

**Asenkron okumalı küçük tamponlar.** LUTRAM, okuma adresi verildiği çevrimde veriyi kombinasyonel olarak verir. Bu sayede GEMV, LN ve dikkat birimlerinin durum makineleri adres-veri hizalaması için ek boru hattı aşaması gerektirmez. Bunun bedeli, okuma yolunun saat periyoduna dahil olmasıdır.

**Tek yazma portu.** Artık ve xb tamponlarına iki farklı kaynak yazar (gömme ile artık toplama; dikkat çıktısı ile GELU). Bu kaynaklar zaman içinde çakışmaz. RTL'de her tampon için tek bir yazma portu kombinasyonel olarak seçilir; sentez aracı tamponu tek portlu RAM olarak çıkarabilir.

**Çarpıcı envanteri.** DSP48E1 bloğu 25×18 bitlik işaretli bir çarpıcı, 48 bitlik bir toplayıcı ve boru hattı yazmaçları içerir; bu boyutu aşan çarpımlar birden fazla DSP'ye bölünür.

**Tablo 15.** Tasarımdaki çarpıcılar ve tahmini DSP48E1 kullanımı.

| Yer | İşlem | Operandlar | Adet | Tahmini DSP |
|---|---|---|---|---|
| GEMV şeritleri | $W \cdot x$ | $8 \times 8$ | 8 | 0–8 |
| Requant | $(acc + b) \cdot m_0$ | $33 \times 17$ | 1 | 2 |
| LayerNorm | $d \cdot d$ | $32 \times 32$ | 1 | 3–4 |
| LayerNorm | $d \cdot inv$ | $32 \times 49$ | 1 | 4–6 |
| Dikkat | $q \cdot k$ | $8 \times 8$ | 1 | 0–1 |
| Dikkat | $e \cdot v$ | $17 \times 8$ | 1 | 1 |
| Dikkat | $(m - s) \cdot C_{exp}$ | $32 \times 16$ | 1 | 2 |
| Dikkat | $A \cdot R$ | $32 \times 49$ | 1 | 4–6 |

Toplam tahmin 16–30 DSP'dir; Zynq-7020'de 220 DSP bulunur. Geniş çarpıcıların operand genişlikleri bilinçli olarak güvenli tutulmuştur. Bölüm 13'teki sınırlar gerçek ihtiyacı gösterir ($d$ için 23 bit, $inv$ için yaklaşık 27 bit, $R$ için 18 bit). Genişlikler daraltılırsa DSP sayısı ve kritik yol kısalır. Hedef cihaz xc7z020'de 53 200 LUT, 106 400 FF, 140 RAMB36 ve 220 DSP bulunur. Tasarımın yaklaşık 24 RAMB36'lık ihtiyacı, cihaz BRAM'inin %17'si kadardır.

## 18 Performans analizi

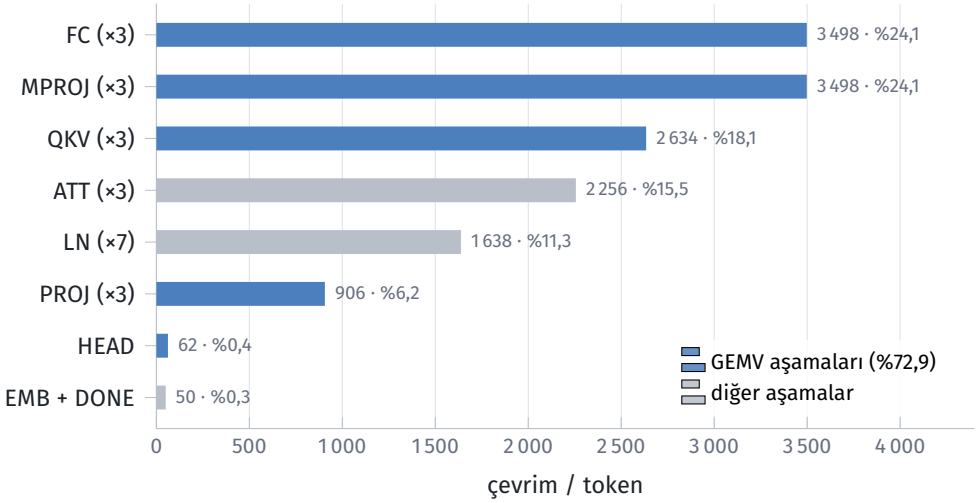
Bir token ortalama 14 544 çevrimde işlenir. Bu, 100 MHz'de 145 µs/token, yani saniyede yaklaşık 6 900 token demektir. Sürenin %72,9'u GEMV aşamalarında geçer (Şekil 13).

MLP'nin iki katmanı (FC ve MPROJ) tek başına sürenin yaklaşık yarısını alır; bu oran Bölüm 8'deki parametre dağılımıyla ($8C^2/12C^2$) uyumludur. Dikkat (%15,5) ve LayerNorm (%11,3) ardışık birimler oldukları için işlem paylarından (%4'ün altında) daha fazla süre alır.

**Analitik çevrim modeli.** Bölüm 15 ve 16'daki çevrim sayıları birleştirilince, $t$ konumundaki bir tokenın süresi kapalı biçimde yazılabilir:

$$\begin{aligned} T_{token}(t) &= \underbrace{49}_{\text{EMB}} + \underbrace{7 \cdot 234}_{\text{LN}} + \underbrace{3\,(878 + 302 + 1166 + 1166) + 62}_{\text{GEMV} = 10\,598} + \underbrace{9\,(33\,(t+1) + 53)}_{\text{dikkat}} + 1 \\ &\approx 12\,763 + 297\,(t+1). \end{aligned} \tag{37}$$

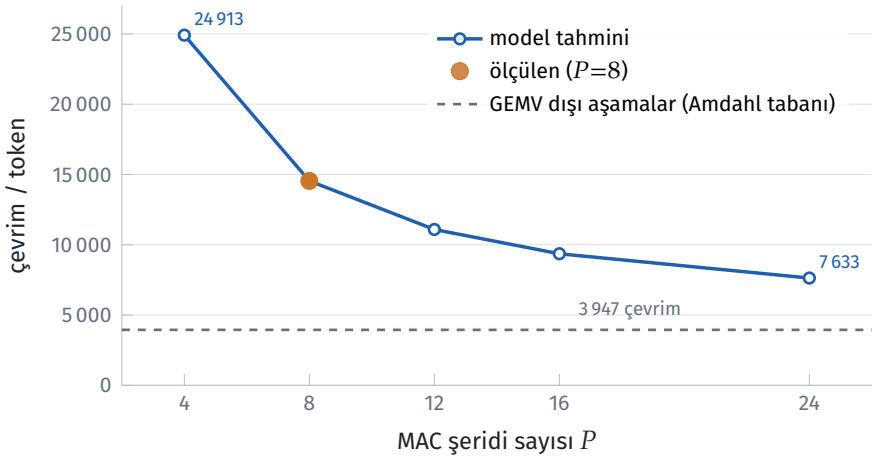
Model $t = 0$ için 13 060, $t = 10$ için 16 030 çevrim verir. Ortalaması 14 545 çevrimdir ve ölçülen 14 544 değeriyle bir çevrim farkla uyuşur.




**Şekil 13.** Token başına çevrim profili (tüm katmanlar toplamı; Verilator, 729 dizi × 11 adım ortalaması). Testbench el sıkışmasıyla birlikte toplam 14 544 çevrimdir.

**Darboğaz ve Amdahl sınırı.** Decode sırasında her ağırlık token başına bir kez kullanılır; aritmetik yoğunluk ağırlık baytı başına 1 MAC'tir. GEMV hızı bu yüzden çevrim başına okunabilen ağırlık baytıyla sınırlıdır: $P = 8$ şerit ROM'dan çevrim başına 8 bayt okur. GEMV dışındaki aşamalar 3 944 çevrim sürer. GEMV sonsuz hızlı olsa bile token süresi bu değerin altına inemez; yalnızca $P$ artırılarak elde edilebilecek en büyük hızlanma $14\,542/3\,944 \approx 3{,}7\times$'tir.

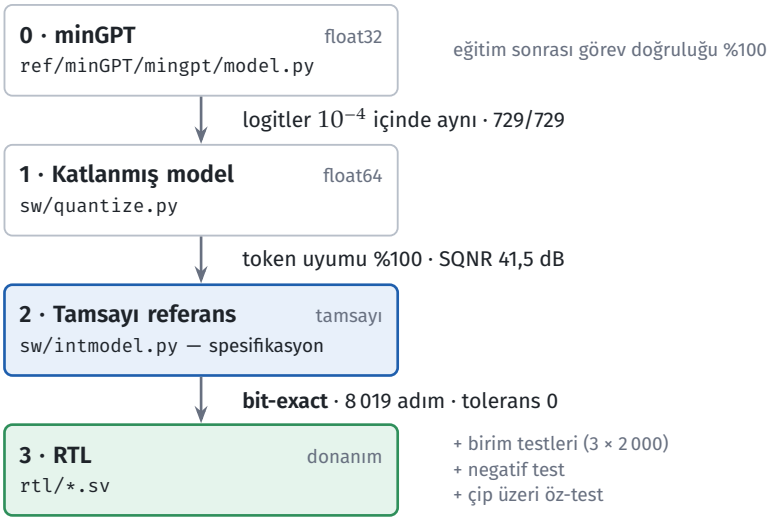
**Paralellik derecesinin etkisi.** $P$, bütün $n_{out}$ değerlerini (144, 48, 192) bölmeli ve $n_{in} > P$ sağlanmalıdır; geçerli değerler 4, 8, 12, 16 ve 24'tür ($P = 32$ geçersizdir, çünkü 144 ve 48, 32'ye bölünmez). Şekil 14, analitik modelin tahminlerini gösterir: $P$'yi 8'den 24'e çıkarmak süreyi yalnızca 1,9 kat kısaltır. Daha fazla hızlanma için LayerNorm ve dikkat birimlerinin de paralelleştirilmesi gerekir.




**Şekil 14.** MAC şeridi sayısı $P$'nin token süresine etkisi (analitik model). $P = 8$ ölçülmüştür. Kesikli çizgi, GEMV dışındaki aşamaların oluşturduğu Amdahl tabanıdır.

## 19 Doğrulama metodolojisi

Donanımın doğruluğu, her biri bir öncekine karşı kontrol edilen dört referans seviyesiyle gösterilmiştir (Şekil 15). Son seviyede RTL, tamsayı referans modelle 8 019 decode adımının tamamında bit düzeyinde aynıdır.




**Şekil 15.** Referans model hiyerarşisi. Seviye 2, donanımın bit düzeyindeki spesifikasyonudur; seviye 2 ile 3 arasındaki karşılaştırma toleranssızdır.

Python tamsayıları sınırsız hassasiyetlidir. Bu yüzden RTL'deki her taşma, yanlış bit genişliği veya farklı bir yuvarlama, ilk etkilediği değerde bir uyumsuzluk olarak görünür.

**Birim testleri** (`tb/tb_arith.sv`). Aritmetik bloklar referans modele ihtiyaç duymayan matematiksel koşullarla test edilir: `div_seq` için $q\,d \le n < (q+1)\,d$, `isqrt_seq` için $r^2 \le n < (r+1)^2$ ve `requant` için davranışsal referansa eşitlik. Her blok 2 000 rastgele girdiyle sınanmıştır.

**Sistem testi** (`tb/tb_gpt_top.sv`). `sw/export.py`, tamsayı modelin 729 girdinin her biri için yaptığı 11 decode adımını `mem/tv.txt` dosyasına yazar. Testbench:

1. **kapalı döngü** çalışır: ilk 6 adımda girdi tokenları verilir, sonraki adımlarda DUT'nin kendi ürettiği token geri beslenir;
2. her adımda token, bütün logitler ve 48 artık değer referansla birebir karşılaştırılır;
3. dizi sonunda üretilen 6 tokenın girdinin sıralı hâli olup olmadığını ayrıca kontrol eder;
4. FSM durumlarına göre çevrim profilini ölçer (Şekil 13).

**Testin testi.** Her zaman PASS veren bir testbench hiçbir şeyi kanıtlamaz. Bu yüzden altın dosyaya bilerek iki hata eklenmiştir: bir artık değere +1 ve değiştirilmiş bir token. Testbench ikisini de doğru dizi ve adımla raporlamış ve FAIL vermiştir.

**Tablo 16.** Sistem testi sonuçları (Verilator 5.052).

| Ölçüt | Değer |
|---|---|
| Kontrol edilen decode adımı | 8 019 |
| Bit-exact diziler | 729 / 729 |
| Doğru sıralanan diziler | 729 / 729 |
| Uyumsuz değer | 0 |

**Karşılaşılan bir SystemVerilog tuzağı.** Bir `initial` bloğunun içindeki döngüde ilk değerle tanımlanan değişkenler (`int x = $urandom();`) varsayılan olarak *static* ömürlüdür ve ilk değer yalnızca bir kez atanır (IEEE 1800, §6.21). Bu hatayla 2 000 rastgele test aslında aynı girdiyi 2 000 kez denerdi. Verilator'ın `IMPLICITSTATIC` uyarısıyla fark edilmiş ve değişkenler `automatic` yapılarak düzeltilmiştir.

**Çip üzeri öz-test** (`tb/tb_selftest.sv`). Bölüm 20'deki öz-test modülü simülasyonda iki kez çalıştırılmıştır: normal modda 729/729 dizi geçmiş, hata enjeksiyonu açıkken 729/729 dizi kalmıştır. Bir çalıştırma 116 637 813 çevrim sürer.

## 20 PYNQ-Z2 entegrasyonu

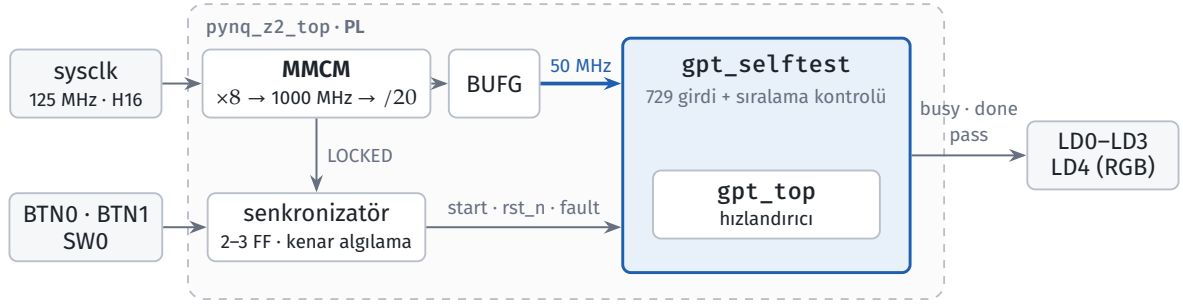
Hedef kart, Zynq-7020 (xc7z020clg400-1) içeren TUL PYNQ-Z2'dir. İlk kart hedefi, PC veya yazılım gerektirmeyen bir **çip üzeri öz-testtir**: FPGA 729 sıralama probleminin hepsini kendisi üretir, çözer ve kontrol eder; sonucu LED ile gösterir.

**Tablo 17.** Aday kartların karşılaştırması.

| Kart | FPGA | RAMB36 | DSP | Değerlendirme |
|---|---|---|---|---|
| **PYNQ-Z2** | Zynq-7020 | 140 | 220 | **seçildi**: 2×Cortex-A9 + PYNQ (Python) |
| Basys 3 | Artix-7 35T | 50 | 90 | sığar, ancak büyümeye yer az; işlemci yok |
| Zybo Z7-20 | Zynq-7020 | 140 | 220 | aynı çip; resmî PYNQ desteği yok |
| Kria KV260 | Zynq US+ K26 | 144 (+64 URAM) | 1 248 | bu boyut için gereğinden büyük |

PYNQ-Z2'nin seçilme nedenleri şunlardır: BRAM'in yalnızca yaklaşık %17'sinin kullanılması, ücretsiz Vivado ML Standard sürümüyle desteklenmesi ve kart üzerindeki ARM işlemci. Bu işlemci, sonraki aşamada hızlandırıcıyı Python'dan sürmeye ve aynı karttaki CPU ile karşılaştırmaya imkân verir.

**Öz-test modülü** (`rtl/gpt_selftest.sv`). Modül, girdileri tabanı 3 olan 6 basamaklı bir sayaçla (odometre) üretir. Her girdi için hızlandırıcıyla 11 adımlık kapalı döngü üretim yapar. Üretilen 6 token iki koşulla kontrol edilir: *monotonluk* (her token bir öncekinden küçük olmamalı) ve *histogram eşitliği* (her rakamın çıktıdaki sayısı girdidekine eşit olmalı). Bu iki koşul birlikte "çıktı, girdinin sıralı hâlidir" önermesine denktir; çipte altın veri saklanmaz. Hata enjeksiyonu açıkken her dizinin ilk çıktısı farklı geçerli bir tokenla değiştirilir. Histogram bu durumda kesinlikle değişeceği için her dizi kalmalıdır.




**Şekil 16.** PYNQ-Z2 üst modülü. 125 MHz kart saatinden MMCM ile 50 MHz üretilir. Butonlar senkronize edilir, öz-test sonucu LED'lerde gösterilir.

**Saat ve reset.** Kart, PL'ye H16 pininden 125 MHz saat verir. Bu saatten bir MMCM ile tasarım saati üretilir:

$$f_{VCO} = 125\ \text{MHz} \times 8 = 1000\ \text{MHz}, \qquad f_{clk} = f_{VCO}/20 = 50\ \text{MHz}. \tag{38}$$

VCO frekansı, -1 hız sınıfı 7-serisi MMCM'nin 600–1200 MHz aralığındadır. 50 MHz, zamanlama raporu görülene kadar bilinçli olarak temkinli seçilmiştir; rapor uygunsa bölen 10'a düşürülerek 100 MHz'e çıkılabilir. PL'ye bağlı bir reset pini yoktur; reset MMCM'nin `LOCKED` sinyali ve BTN1 ile üretilir.

**Tablo 18.** Kart arayüzü.

| Öğe | Anlamı |
|---|---|
| LD0 | yanıp söner: bitstream çalışıyor |
| LD1 / LD2 | test sürüyor / bitti |
| LD3 | hata enjeksiyonu açık (SW0) |
| LD4 (RGB) | mavi: sürüyor · yeşil: 729/729 doğru · kırmızı: hata var |
| BTN0 / BTN1 / SW0 | testi yeniden başlat / reset / hata enjeksiyonu |

Bir çalıştırma 116,6 milyon çevrim sürer: 50 MHz'de yaklaşık 2,3 s, 100 MHz'de 1,17 s. Kart akışı iki komuttan oluşur: `scripts/vivado_build_pynq_z2.tcl` sentez, yerleştirme, yönlendirme ve bitstream üretimini yapar ve kaynak, zamanlama, güç raporlarını yazar; `scripts/vivado_program.tcl` kartı JTAG üzerinden programlar. Beklenen sonuç: LD4 yaklaşık 2,3 s mavi yanar, sonra yeşil olur. Bu adımlar hazırdır, ancak raporun tarihinde henüz çalıştırılmamıştır.

## 21 Sınırlamalar ve gelecek çalışmalar

Mevcut tasarım, işlevsel doğruluğu kanıtlanmış bir temel sürümdür. Bir sonraki adım onu kart üzerinde çalıştırmak, ardından ölçerek iyileştirmektir.

**Sınırlamalar.**

- **Fiziksel doğrulama yok:** sentez, zamanlama kapanışı ve kart üzerinde çalıştırma henüz yapılmamıştır. 100 MHz bir hedeftir, ölçülmüş değildir.

- **Model boyutu:** model küçüktür (85 728 parametre) ve dar bir görevde eğitilmiştir; metin üreten bir dil modeli değildir.
- **Ölçek:** bütün ağırlıkların BRAM'e sığması varsayılmıştır. Bu varsayım Zynq-7020'de yaklaşık 600 bin INT8 parametreye kadar geçerlidir.
- **Kod çözme:** yalnızca açgözlü kod çözme vardır; sıcaklıklı örnekleme ve top-k yoktur.
- **Bit genişlikleri:** geniş çarpıcıların genişlikleri güvenli tarafta tutulmuştur; kritik yol ve DSP sayısı henüz optimize edilmemiştir.

**Gelecek çalışmalar.**

1. **Kartta çalıştırma ve ölçüm:** PYNQ-Z2 bitstream'i, kaynak/Fmax/güç raporları ve yeşil LED.
2. **Paralellik taraması:** $P \in \{4, 8, 12, 16, 24\}$ için çevrim, BRAM, DSP ve Fmax değişimi (Şekil 14). $P$, `quantize.py` içinde tek bir parametredir.
3. **Tek geçişli LayerNorm:** $\sum d^2 = N(N\sum x^2 - s^2)$ özdeşliğiyle LN başına 48 çevrim kazancı.
4. **Paralel dikkat:** SCORE ve AV aşamalarında $d_h = 16$ boyutu üzerinden paralel MAC; dikkat süresinde yaklaşık 16 kat azalma.
5. **Bit genişliği analizi:** Bölüm 13'teki sınırlardan türetilen dar operandlar ve boru hattı bölme ile daha yüksek Fmax.
6. **PS entegrasyonu:** AXI4-Lite sarmalayıcı, Vivado blok tasarımı ve PYNQ Python sürücüsü. Böylece aynı karttaki ARM işlemciyle (NumPy/PyTorch) gecikme ve enerji karşılaştırması yapılabilir.
7. **Daha büyük görev:** minGPT'nin `projects/chargpt` örneğiyle karakter düzeyinde metin üretimi. Mimari aynı kalır; büyük modellerde ağırlıklar DDR'dan akış hâlinde okunur.

## Kaynaklar


[1] A. Vaswani, N. Shazeer, N. Parmar ve ark., "Attention Is All You Need," *Advances in Neural Information Processing Systems* (*NeurIPS*), 2017.

[2] A. Radford, K. Narasimhan, T. Salimans, I. Sutskever, "Improving Language Understanding by Generative Pre-Training," OpenAI, 2018.

[3] A. Radford, J. Wu, R. Child ve ark., "Language Models are Unsupervised Multitask Learners," OpenAI, 2019.

[4] T. Brown, B. Mann, N. Ryder ve ark., "Language Models are Few-Shot Learners," *NeurIPS*, 2020.

[5] A. Karpathy, *minGPT*, `https://github.com/karpathy/minGPT`.

[6] K. He, X. Zhang, S. Ren, J. Sun, "Deep Residual Learning for Image Recognition," *CVPR*, 2016.

[7] J. L. Ba, J. R. Kiros, G. E. Hinton, "Layer Normalization," arXiv:1607.06450, 2016.

[8] R. Xiong, Y. Yang, D. He ve ark., "On Layer Normalization in the Transformer Architecture," *ICML*, 2020.

[9] D. Hendrycks, K. Gimpel, "Gaussian Error Linear Units (GELUs)," arXiv:1606.08415, 2016.

[10] I. Loshchilov, F. Hutter, "Decoupled Weight Decay Regularization," *ICLR*, 2019.

[11] B. Jacob, S. Kligys, B. Chen ve ark., "Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference," *CVPR*, 2018.

[12] M. Nagel, M. Fournarakis, R. A. Amjad ve ark., "A White Paper on Neural Network Quantization," arXiv:2106.08295, 2021.

[13] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, K. Keutzer, "I-BERT: Integer-only BERT Quantization," *ICML*, 2021.

[14] P. A. Samuelson, "How Deviant Can You Be?," *Journal of the American Statistical Association*, 63(324), 1968.

[15] V. Sze, Y.-H. Chen, T.-J. Yang, J. S. Emer, "Efficient Processing of Deep Neural Networks: A Tutorial and Survey," *Proceedings of the IEEE*, 105(12), 2017.

[16] R. Pope, S. Douglas, A. Chowdhery ve ark., "Efficiently Scaling Transformer Inference," *MLSys*, 2023.

[17] B. Li, S. Pandey, H. Fang ve ark., "FTRANS: Energy-Efficient Acceleration of Transformers using FPGA," *ISLPED*, 2020.

[18] T. J. Ham, S. J. Jung, S. Kim ve ark., "$A^3$: Accelerating Attention Mechanisms in Neural Networks with Approximation," *HPCA*, 2020.